TEMA 2: SISTEMAS ALGORÍTMICOS PROGRAMABLES

2.1.- Introducción: Sistemas digitales de proceso.

2.2.- Arquitectura de un ordenador sencillo.

2.3.- Unidad Aritmético-Lógica y Desplazador.

2.4.- Control cableado de ciclo sencillo – Arquitectura Harvard.

2.5.- Ruta de datos y control en canalización (pipeline).

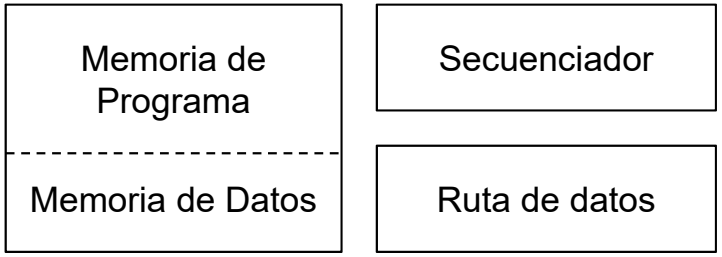
2.6.- Control microprogramado de ciclos múltiples – Arquitectura Von-Neumann.

TEMA 2: SISTEMAS ALGORÍTMICOS PROGRAMABLES

2.1.- Introducción: Sistemas digitales de proceso

Los sistemas digitales vistos en el tema anterior sólo podían llevar a cabo una determinada operación. En este tema estudiaremos los *sistemas digitales de proceso programables*, también llamados *procesadores* o *CPUs* (Central Processing Units), capaces de realizar diferentes tipos de operaciones.

Elementos en un sistema digital de proceso programable:



La operación concreta a realizar se especifica mediante un *programa*, el cual consta de una serie de *instrucciones* que, en su conjunto, llevan a cabo la operación deseada. Tanto el programa como los datos sobre los que opera, se almacenan en circuitos de memoria.

La unidad de control del sistema consta de un secuenciador que gobierna el orden en que se van ejecutando las instrucciones del programa.

La unidad de proceso del sistema (ruta de datos) consta de una serie de registros así como de unidades funcionales que permiten la manipulación de los datos del programa.

Tipos de sistemas digitales de proceso programables:

- Microprocesadores: sistemas de propósito general, núcleo central de los ordenadores.
- Microcontroladores: especializados en el control de procesos en tiempo real. Más sencillos y más rápidos, suelen incluir dispositivos periféricos.
- Procesadores digitales de señal (DSPs): optimizados para realizar operaciones numéricas a muy alta velocidad. Son especialmente útiles para el procesado y representación de señales en tiempo real.

INSTRUCCIONES

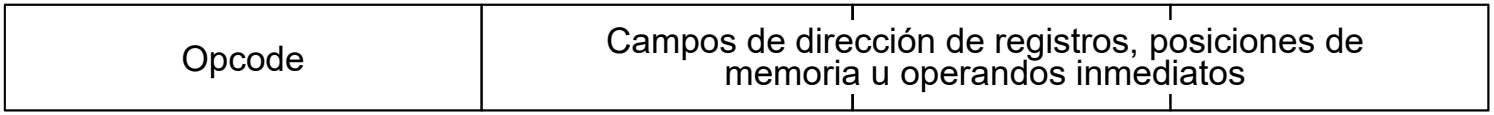
El usuario le dice al procesador las operaciones a realizar y su secuencia mediante un *programa*, el cual consiste en una lista de *instrucciones* que especifica las operaciones, los operandos y en qué orden se van procesando.

Las instrucciones del programa y los datos se almacenan en memoria RAM, ya sea en chips separados (arquitectura Harvard) o en el mismo chip (arquitectura Von-Neumann).

La unidad de control lee una instrucción de la memoria, la decodifica, y después la lleva a cabo ejecutando una o varias microoperaciones sobre la unidad de proceso (ruta de datos).

Internamente, la instrucción es una colección de bits que indica al procesador la operación a realizar, dónde se encuentran los operandos, dónde guardar el resultado de la operación, y en ocasiones, dónde se encuentra la siguiente instrucción a ejecutar.

El formato de la instrucción normalmente se especifica mediante un rectángulo que simboliza sus bits tal y como aparecen en las posiciones de memoria o en registros de control:



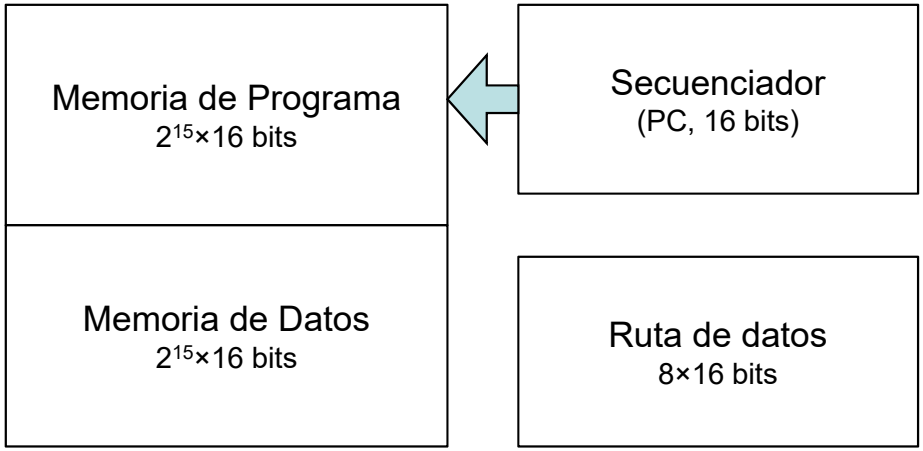
El *código de operación* (opcode) identifica la instrucción. A la colección de todas las instrucciones que puede ejecutar un procesador se le denomina *conjunto de instrucciones*, y a la completa descripción de dicho conjunto, *arquitectura del conjunto de instrucciones*.

El resto de los campos especifica dónde encontrar los operandos y en qué lugar colocar el resultado de la operación (registros, posiciones en la memoria u operandos inmediatos).

FORMATO DE LA INSTRUCCIÓN

Consideraremos un procesador con un conjunto de 128 instrucciones, como máximo, y que dispone de 8 registros (R0 a R7) de 16 bits en su ruta de datos. La memoria tiene una capacidad de 2^{16}×16 bits.

El secuenciador consiste en un registro, llamado *contador de programa* PC, cuyo contenido apunta a la posición de memoria donde se encuentra la instrucción a ejecutar.



Ejemplos de formato de instrucción:

a) Registro:

15 … 9	8 … 6	5 … 3	2 … 0
Opcode	Registro de destino (DR)	Registro fuente A (SA)	Registro fuente B (SB)

b) Inmediato:

15 … 9	8 … 6	5 … 3	2 … 0
Opcode	Registro de destino (DR)	Registro fuente A (SA)	Operando (OP)

c) Salto:

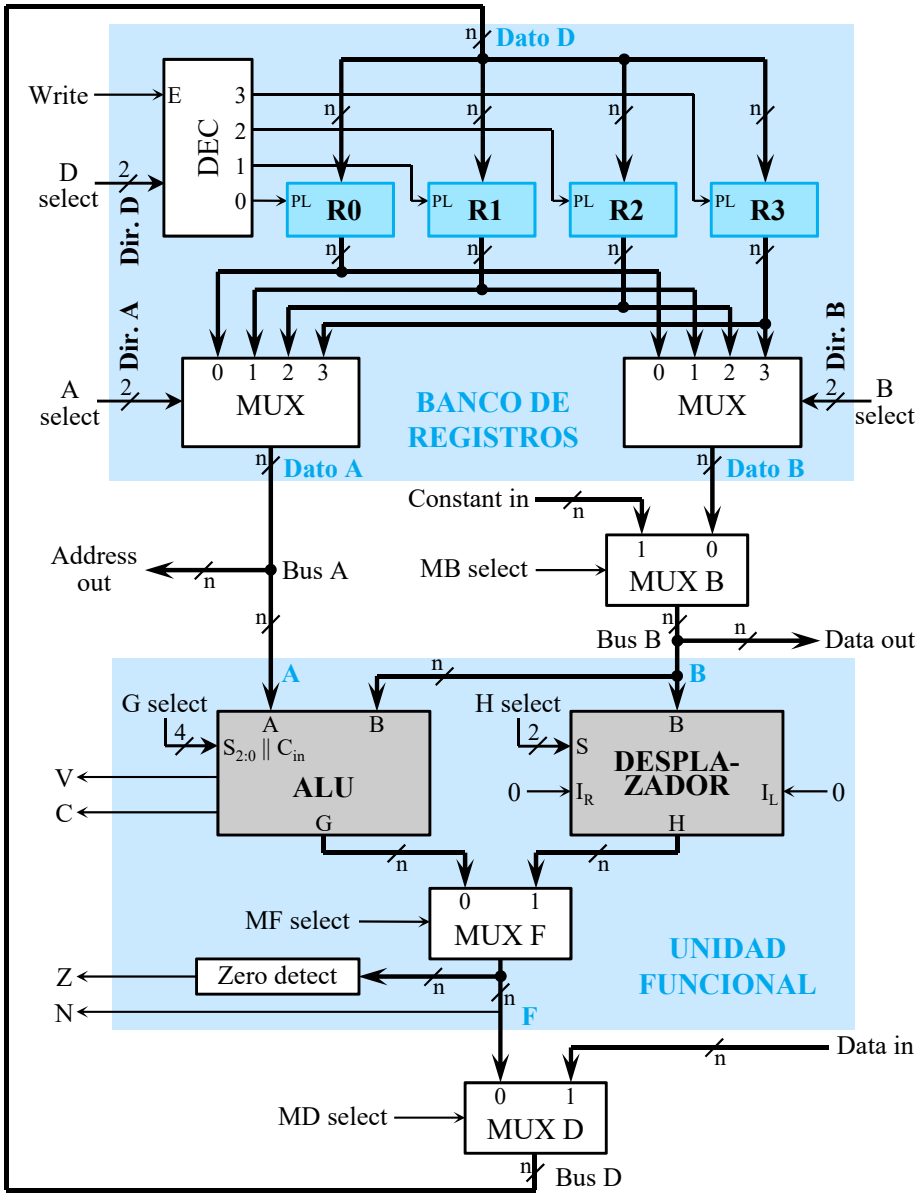
15 … 9	8 … 6	5 … 3	2 … 0
Opcode	Dirección msb (AD)	Registro fuente A (SA)	Dirección lsb (AD)

RUTA DE DATOS

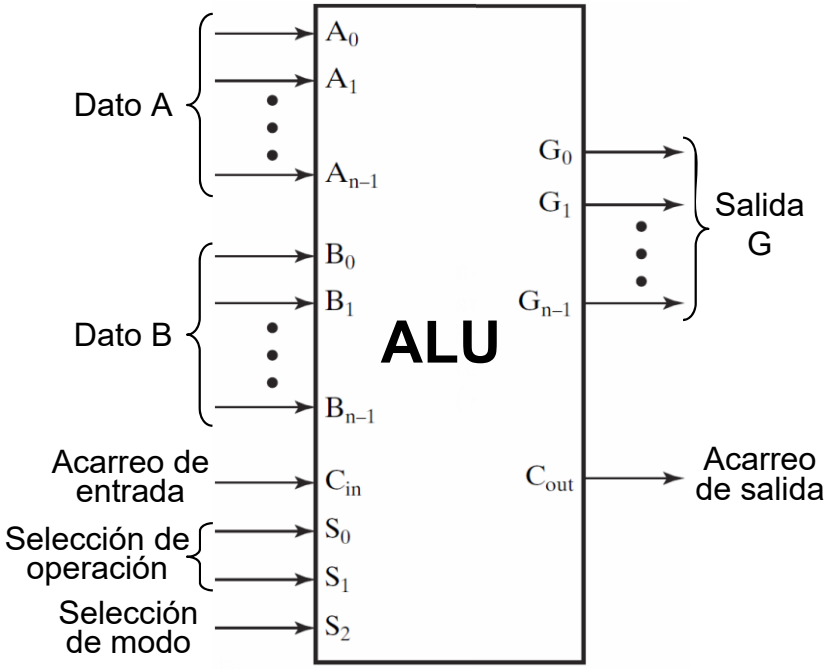
Los sistemas algorítmicos programables emplean un cierto número de registros de almacenamiento conjuntamente con una unidad funcional.

Una microoperación consiste en aplicar a la unidad funcional los contenidos de dos registros fuente, hacer una operación, y almacenar el resultado en un registro destino (en un solo ciclo de reloj):

- En el bus A se puede leer el contenido de cualquier registro.
- En el bus B se puede leer el contenido de un registro o una palabra externa *Constant in*.
- Con G se selecciona la operación a realizar en la ALU. Con H un desplazamiento en el desplazador.
- En el bus D se puede leer la salida de la ALU, la del desplazador, o un dato externo *Data in*.
- Con el decodificador se selecciona el registro de destino.
- Bits de estado de la Unidad Funcional:
  - V: Desbordamiento.
  - C: Acarreo.
  - Z: Cero.
  - N: Negativo.



UNIDAD ARITMÉTICO-LÓGICA



Selección de operación

S_2	S_1	S_0	C_{in}	Operación	Función
0	0	0	0	$G = A$	Transfiere A
0	0	0	1	$G = A + 1$	Incrementa A
0	0	1	0	$G = A + B$	Suma A+B
0	0	1	1	$G = A + B + 1$	Suma A+B+1
0	1	0	0	$G = A + \overline{B}$	Suma A con el complemento a 1 de B
0	1	0	1	$G = A + \overline{B} + 1$	Resta A-B
0	1	1	0	$G = A - 1$	Decrementa A
0	1	1	1	$G = A$	Transfiere A
1	X	0	0	$G = A \wedge B$	AND de A y B
1	X	0	1	$G = A \vee B$	OR de A y B
1	X	1	0	$G = A \oplus B$	XOR de A y B
1	X	1	1	$G = \overline{A}$	NOT de A (complemento a 1 de A)

<u>Características:</u>

- S_2 es un bit que permite seleccionar entre operaciones aritméticas (S_2=0) o lógicas (S_2=1).
- C_{in}, S_1 y S_0 permiten seleccionar entre 8 operaciones aritméticas distintas.
- C_{in} y S_0 permiten seleccionar entre 4 operaciones lógicas distintas.

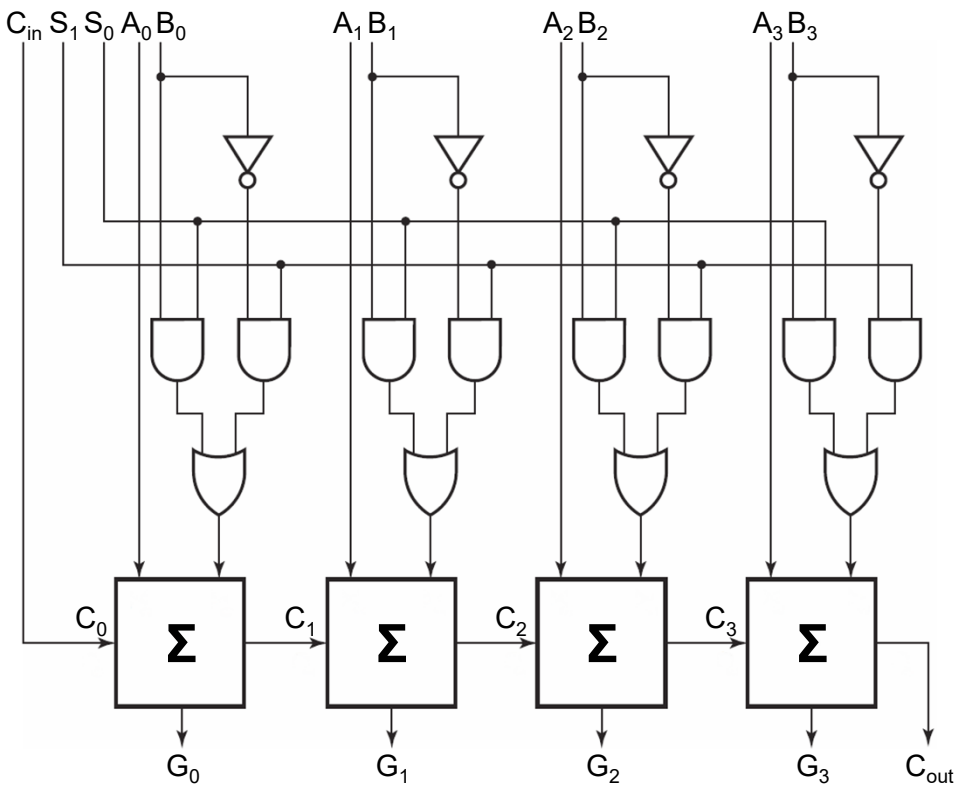
Diseñaremos primeramente la parte aritmética, después la parte lógica, y finalmente las combinaremos en un solo circuito.

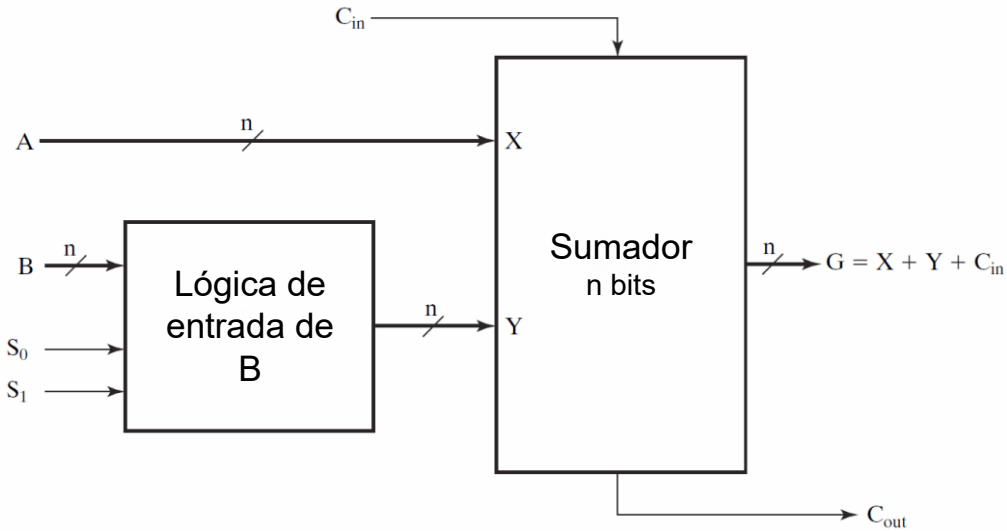
UNIDAD ARITMÉTICO-LÓGICA

BLOQUE ARITMÉTICO

Teniendo en cuenta las operaciones a realizar, el diseño del bloque aritmético se puede plantear:

Selección		Entr.	Operación	
S_1	S_0	Y	$C_{in} = 0$	$C_{in} = 1$
0	0	Todo 0	$G = A$	$G = A + 1$
0	1	B	$G = A + B$	$G = A + B + 1$
1	0	$\overline{B}$	$G = A + \overline{B}$	$G = A + \overline{B} + 1$
1	1	Todo 1	$G = A - 1$	$G = A$





Entradas			Salida	
S_1	S_0	B_i	Y_i	
0	0	0	0	$Y_i = 0$
0	0	1	0	
0	1	0	0	$Y_i = B_i$
0	1	1	1	
1	0	0	1	$Y_i = \overline{B}_i$
1	0	1	0	
1	1	0	1	$Y_i = 1$
1	1	1	1	

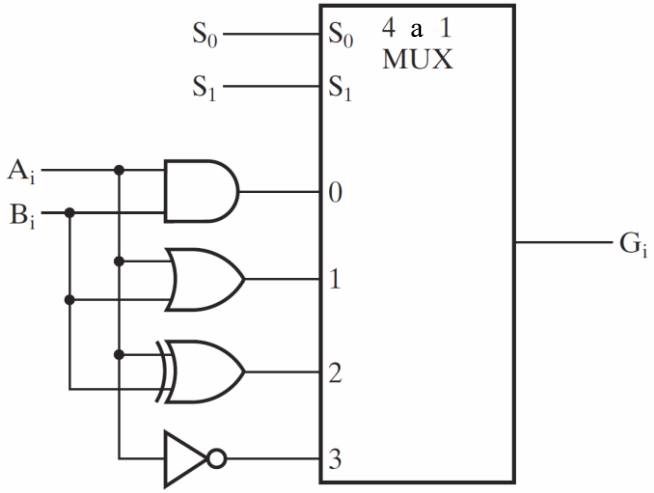
B_i \ S_1S_0	00	01	11	10
0	0	0	1	1
1	0	1	1	0

$$Y_i = B_iS_0 + \overline{B}_iS_1$$

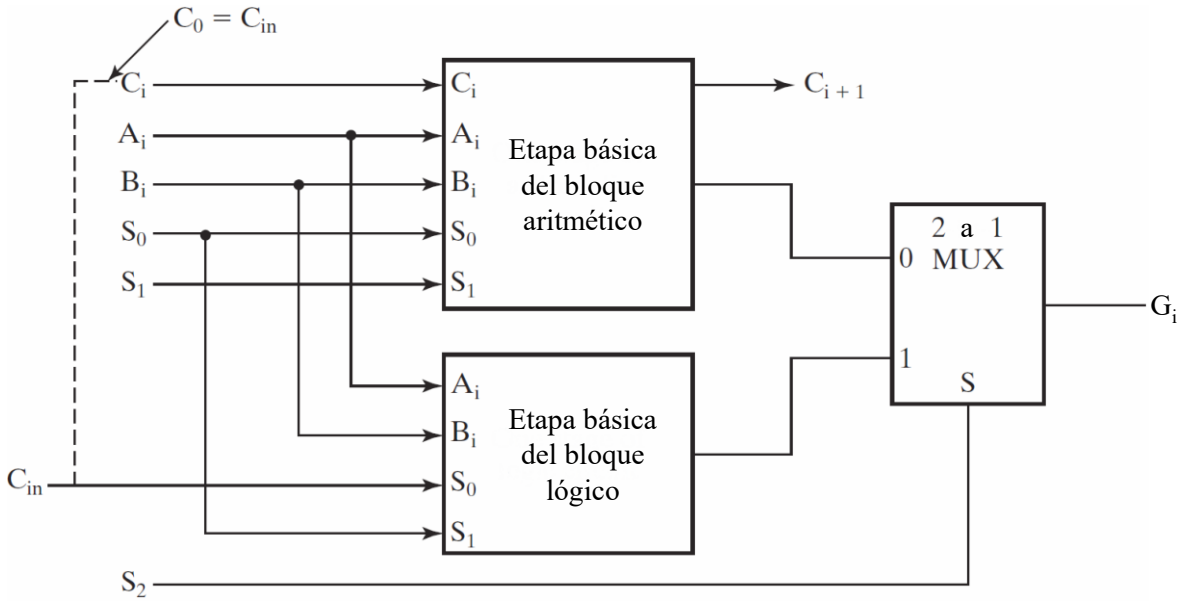
UNIDAD ARITMÉTICO-LÓGICA
BLOQUE LÓGICO Y CIRCUITO FINAL

La operaciones lógicas se hacen bit a bit, con lo que el diseño se puede plantear a partir de un simple multiplexor para cada bit de salida:

S_1	S_0	Salida	Operación
0	0	$G = A \wedge B$	AND
0	1	$G = A \vee B$	OR
1	0	$G = A \oplus B$	XOR
1	1	$G = \overline{A}$	NOT

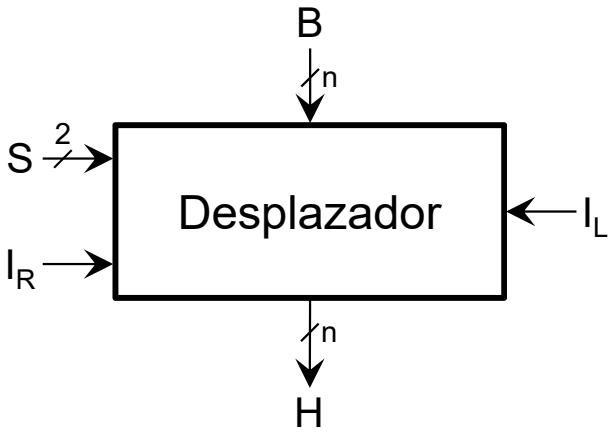


Los bloques aritmético y lógico se pueden combinar utilizando otro multiplexor:



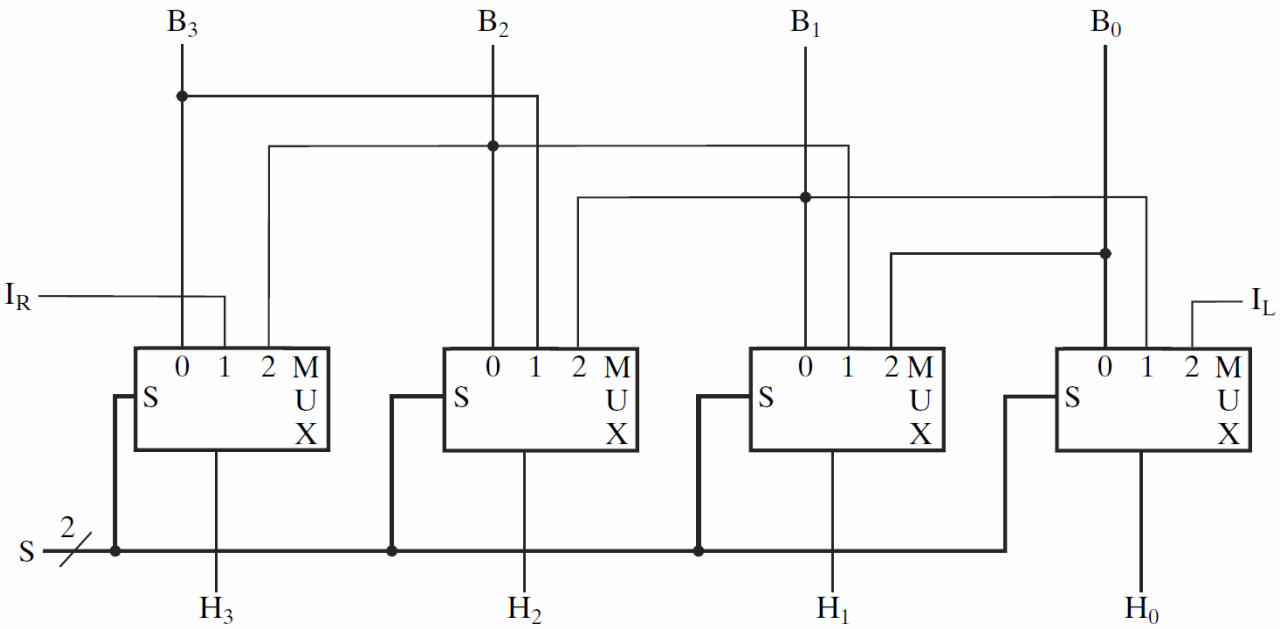
DESPLAZADOR

Plantearemos el diseño como circuito combinacional para que el desplazamiento se pueda hacer en un solo ciclo de reloj.



Características:

- Con S=00 H=B, con S=01 se desplaza un bit a la derecha, y con S=10 se desplaza un bit a la izquierda.
- I_R es un bit que se introduce por la izquierda en un desplazamiento a la derecha.
- I_L es un bit que se introduce por la derecha en un desplazamiento a la izquierda.



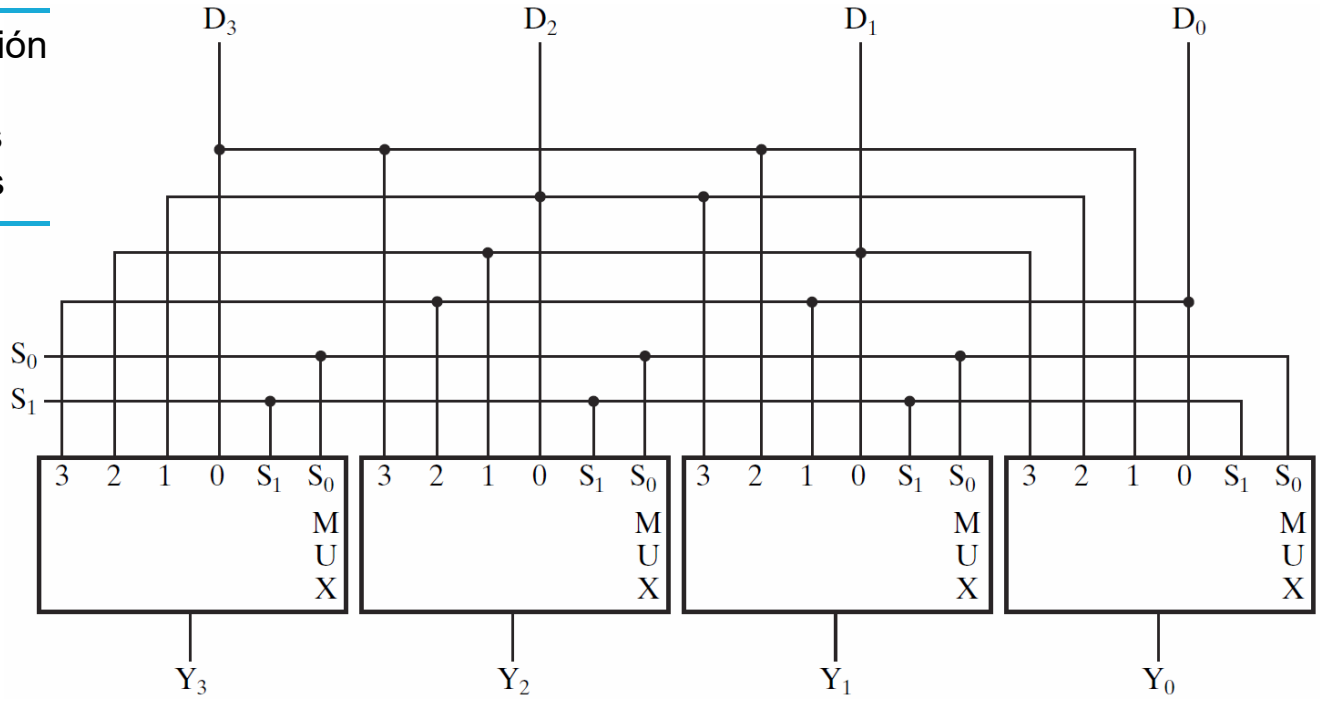
DESPLAZADOR DE BARRIL

En ocasiones resulta útil poder desplazar un número arbitrario de bits en un solo ciclo de reloj.

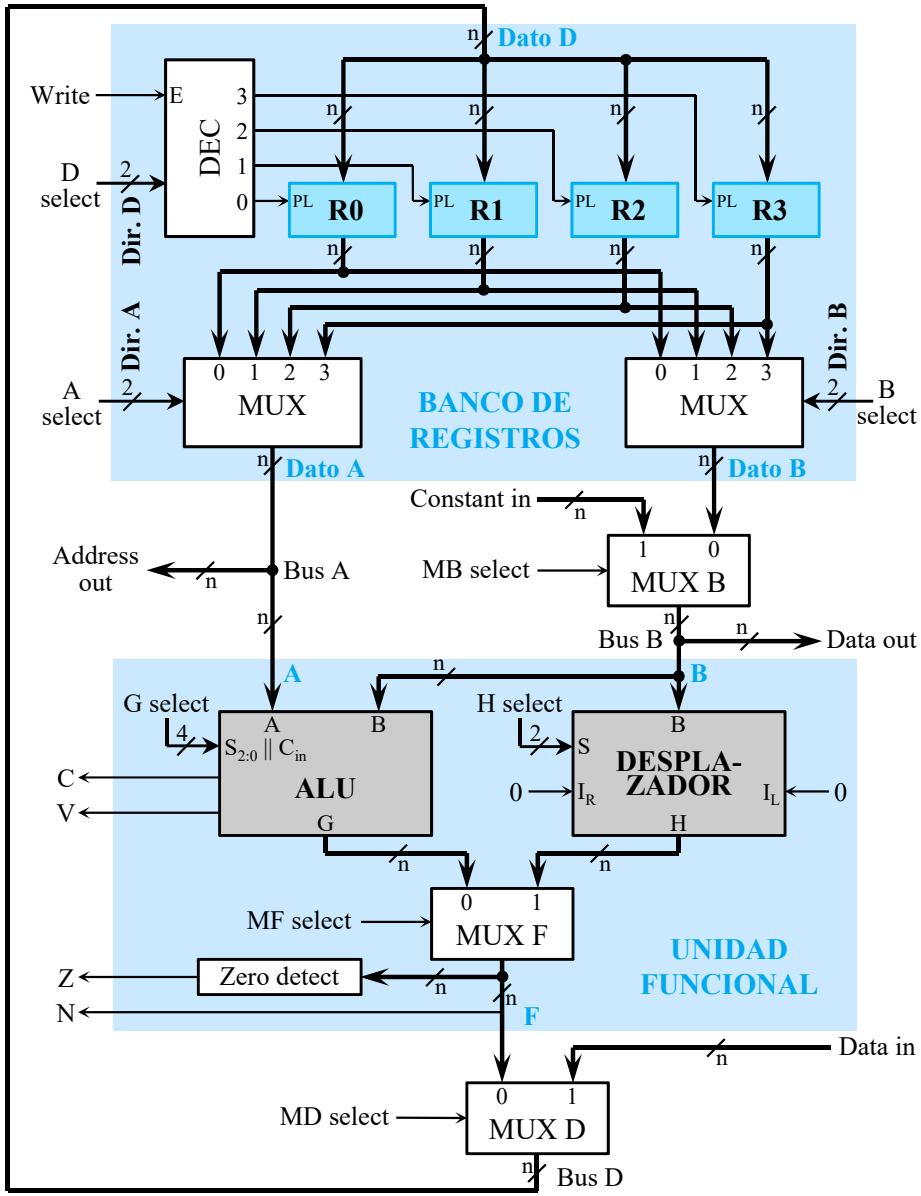
Un desplazador de barril es un circuito combinacional que desplaza (o rota) los bits de la palabra de entrada un cierto número de posiciones especificado mediante un valor binario en unas líneas de selección.

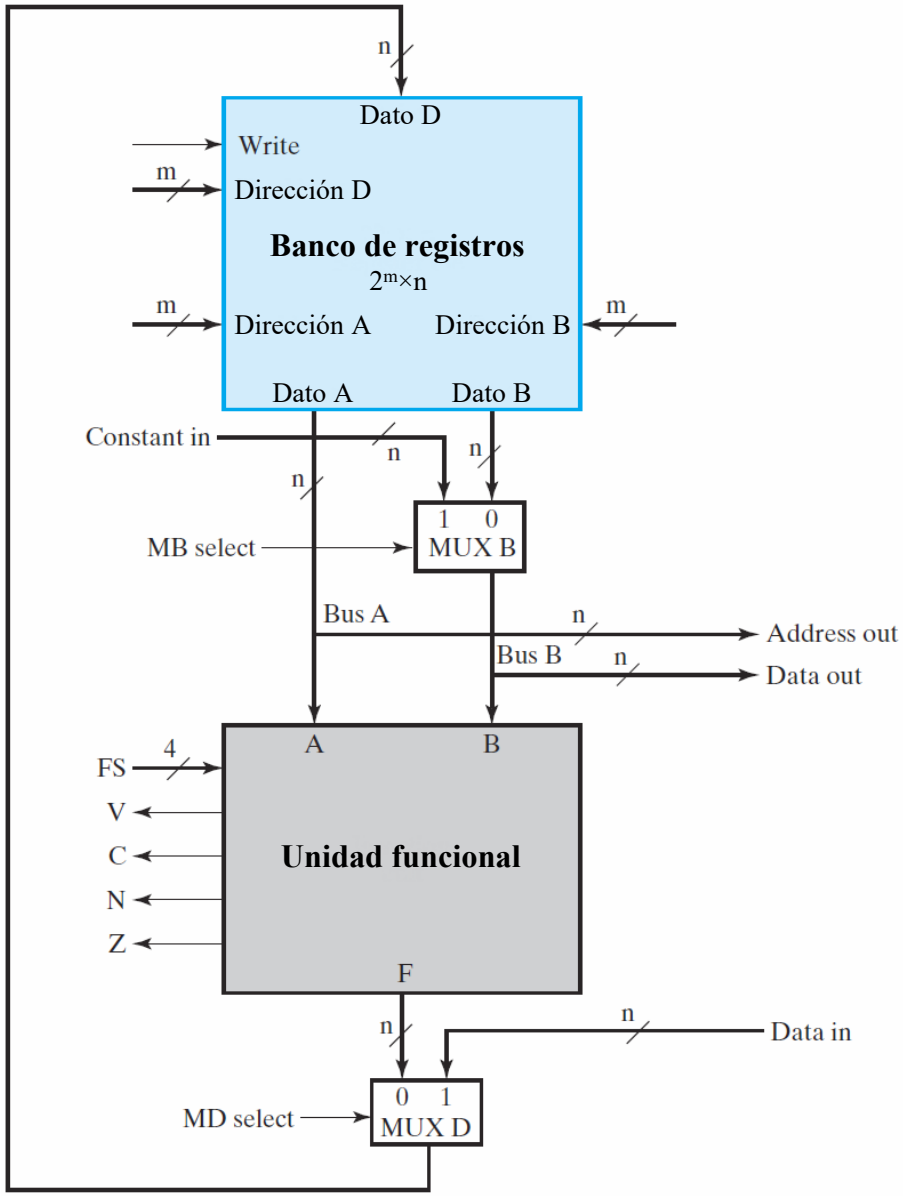
Ejemplo: Desplazador de barril de 4 bits (rotación a la izquierda).

Selec.	**Salidas**	
S_1 S_0	Y_3 Y_2 Y_1 Y_0	Operación
0 0	D_3 D_2 D_1 D_0	No hay rotación
0 1	D_2 D_1 D_0 D_3	Rota un bit
1 0	D_1 D_0 D_3 D_2	Rota dos bits
1 1	D_0 D_3 D_2 D_1	Rota tres bits



REPRESENTACIÓN DE LA RUTA DE DATOS





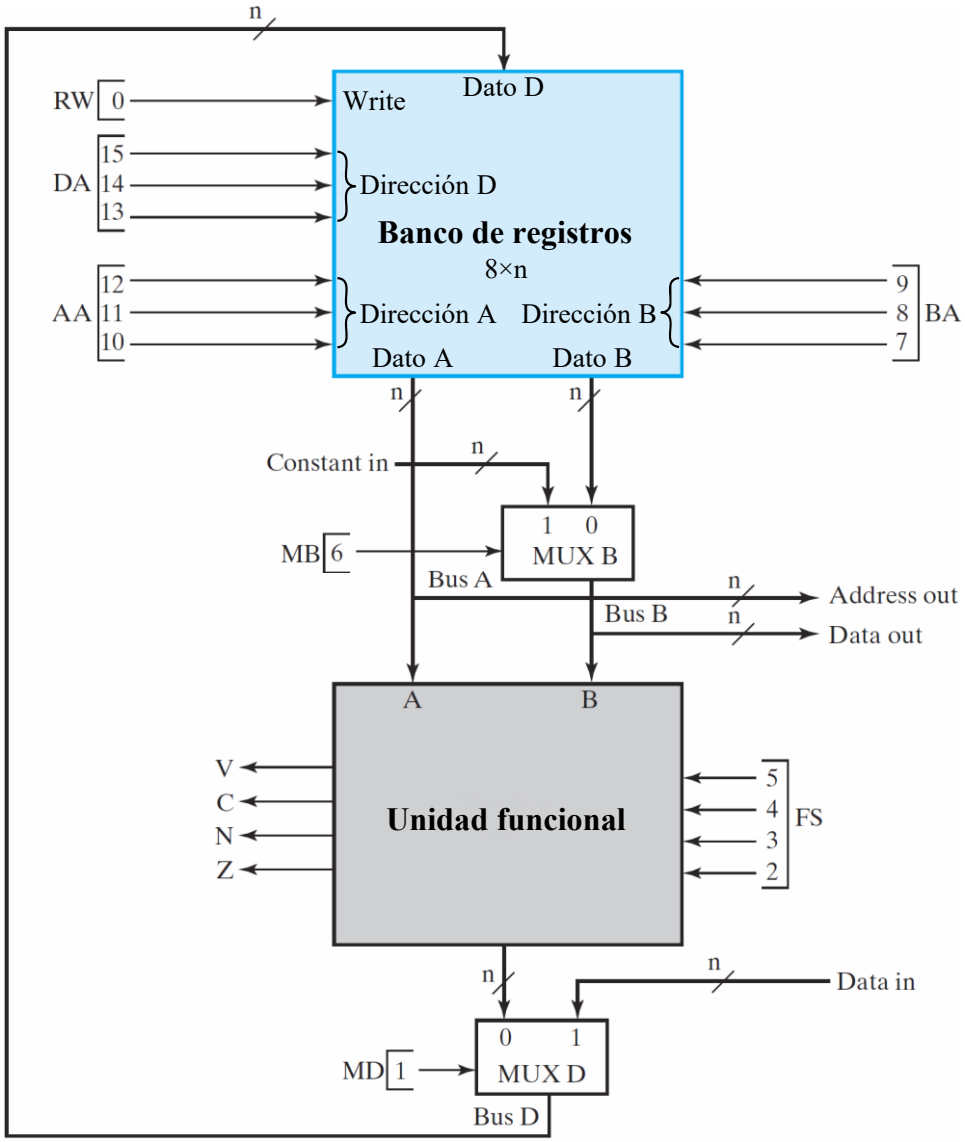
REPRESENTACIÓN DE LA RUTA DE DATOS

Correspondencia entre el código FS (4 bits) y las entradas MF Select (1 bit), G Select (4 bits) y H Select (2 bits):

FS(3:0)	MF Select	G Select(3:0)	H Select(1:0)	Microoperación
0000	0	0000	XX	$F = A$
0001	0	0001	XX	$F = A + 1$
0010	0	0010	XX	$F = A + B$
0011	0	0011	XX	$F = A + B + 1$
0100	0	0100	XX	$F = A + \overline{B}$
0101	0	0101	XX	$F = A + \overline{B} + 1$
0110	0	0110	XX	$F = A - 1$
0111	0	0111	XX	$F = A$
1000	0	1X00	XX	$F = A \wedge B$
1001	0	1X01	XX	$F = A \vee B$
1010	0	1X10	XX	$F = A \oplus B$
1011	0	1X11	XX	$F = \overline{A}$
1100	1	XXXX	00	$F = B$
1101	1	XXXX	01	$F = \text{sr } B$
1110	1	XXXX	10	$F = \text{sl } B$

Se tiene que: $MF=F_3 \cdot F_2$, $G_3=F_3$, $G_2=F_2$, $G_1=F_1$, $G_0=F_0$, $H_1=F_1$ y $H_0=F_0$.

LA PALABRA DE CONTROL



Particularizando para un banco de 8 regitros, se tiene que la palabra de control es de 16 bits:

15 14 13	12 11 10	9 8 7	6	5 4 3 2	1	0
DA	AA	BA	MB	FS	MD	RW

DA,AA,BA		MB		FS		MD		RW	
Reg.	Code	Select	Code	Function	Code	Select	Code	Action	Code
R0	000	Register	0	F=A	0000	FnUt	0	No Write	0
R1	001	Constant	1	F=A+1	0001	Data	1	Write	1
R2	010			F=A+B	0010				
R3	011			F=A+B+1	0011				
R4	100			F=A+$\overline{B}$	0100				
R5	101			F=A+$\overline{B}$+1	0101				
R6	110			F=A-1	0110				
R7	111			F=A	0111				
				F=A∧B	1000				
				F=A∨B	1001				
				F=A⊕B	1010				
				F=$\overline{A}$	1011				
				F=B	1100				
				F=sr B	1101				
				F=sl B	1110				

LA PALABRA DE CONTROL

Ejemplos de microoperaciones en la ruta de datos, usando notación simbólica:

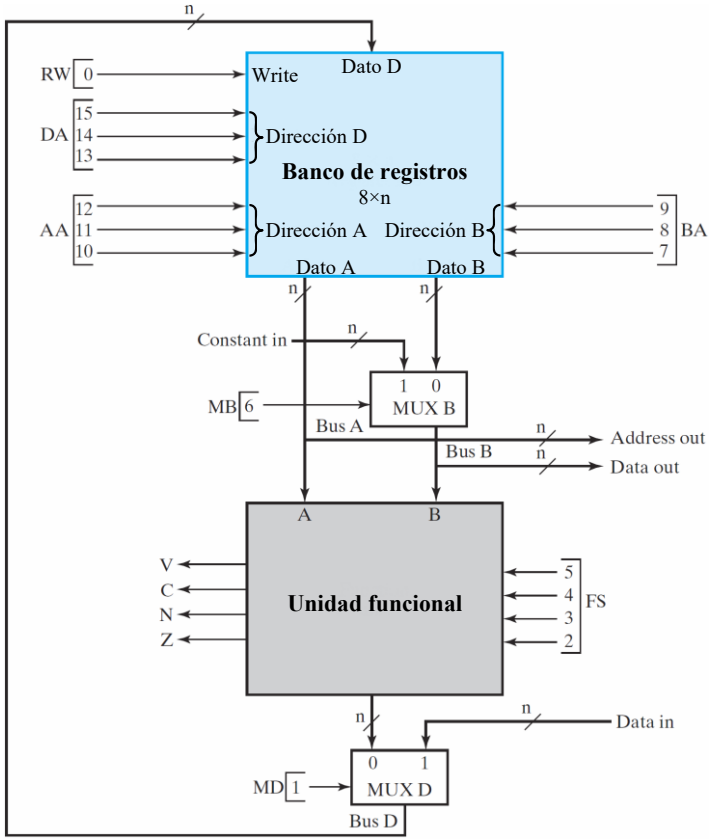
Micro-operación	DA	AA	BA	MB	FS	MD	RW
$R1 \leftarrow R2 - R3$	$R1$	$R2$	$R3$	Register	$F = A + \overline{B} + 1$	FnUt	Write
$R4 \leftarrow$ sl R6	$R4$	—	$R6$	Register	$F =$ sl B	FnUt	Write
$R7 \leftarrow R7 + 1$	$R7$	$R7$	—	—	$F = A + 1$	FnUt	Write
$R1 \leftarrow R0 + 2$	$R1$	$R0$	—	Constant	$F = A + B$	FnUt	Write
Data out $\leftarrow R3$	—	—	$R3$	Register	—	—	No Write
$R4 \leftarrow$ Data in	$R4$	—	—	—	—	Data	Write
$R5 \leftarrow 0$	$R5$	$R5$	$R5$	Register	$F = A \oplus B$	FnUt	Write

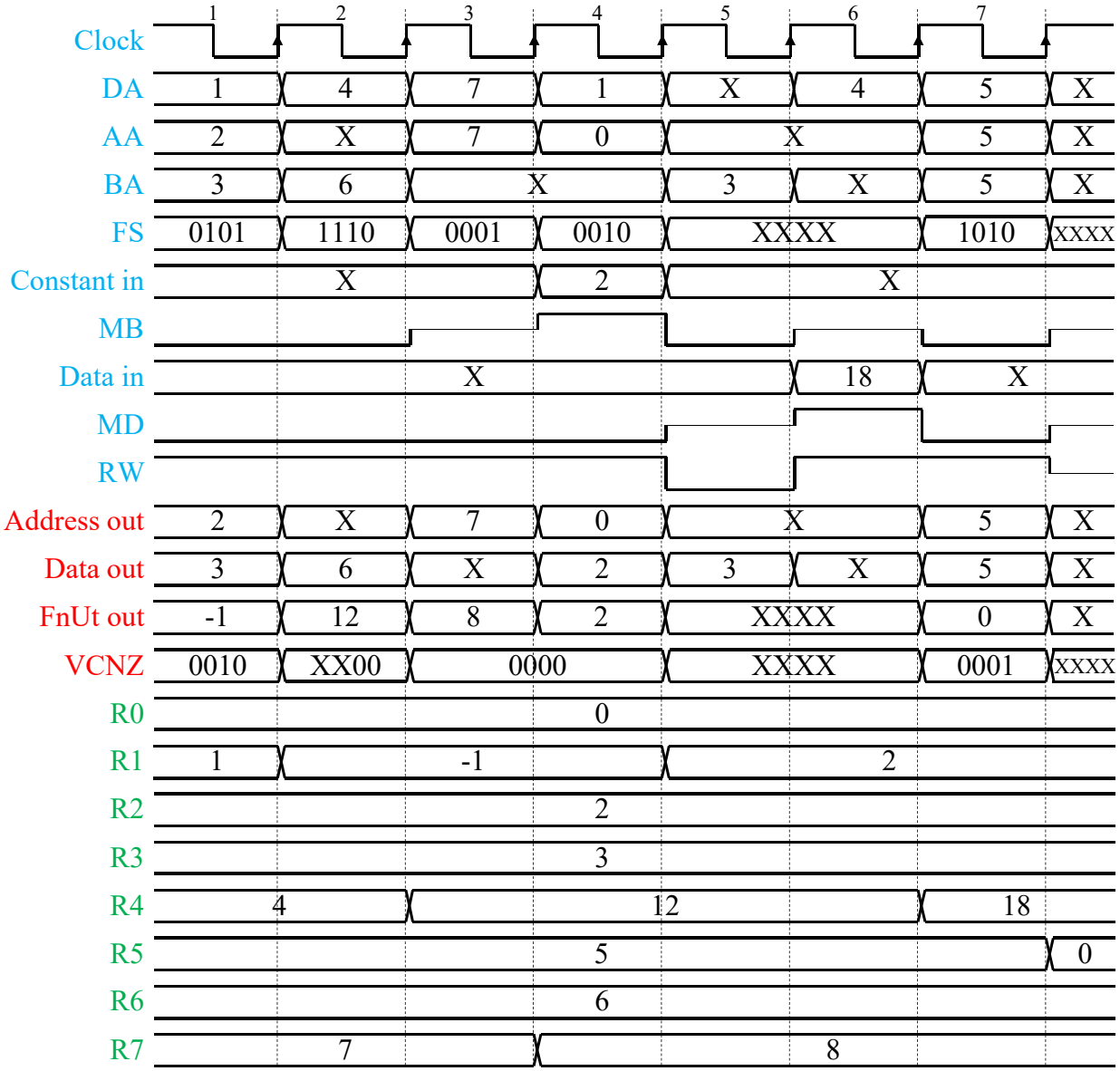
Palabras de control correspondientes a los ejemplos anteriores:

Micro-operación	DA	AA	BA	MB	FS	MD	RW
$R1 \leftarrow R2 - R3$	001	010	011	0	0101	0	1
$R4 \leftarrow$ sl R6	100	XXX	110	0	1110	0	1
$R7 \leftarrow R7 + 1$	111	111	XXX	X	0001	0	1
$R1 \leftarrow R0 + 2$	001	000	XXX	1	0010	0	1
Data out $\leftarrow R3$	XXX	XXX	011	0	XXXX	X	0
$R4 \leftarrow$ Data in	100	XXX	XXX	X	XXXX	1	1
$R5 \leftarrow 0$	101	101	101	0	1010	0	1

TEMPORIZACIÓN DE SEÑALES

Clock	Microoperación
1	$R1 \leftarrow R2 - R3$
2	$R4 \leftarrow$ sl R6
3	$R7 \leftarrow R7 + 1$
4	$R1 \leftarrow R0 + 2$
5	Data out $\leftarrow R3$
6	$R4 \leftarrow$ Data in
7	$R5 \leftarrow 0$



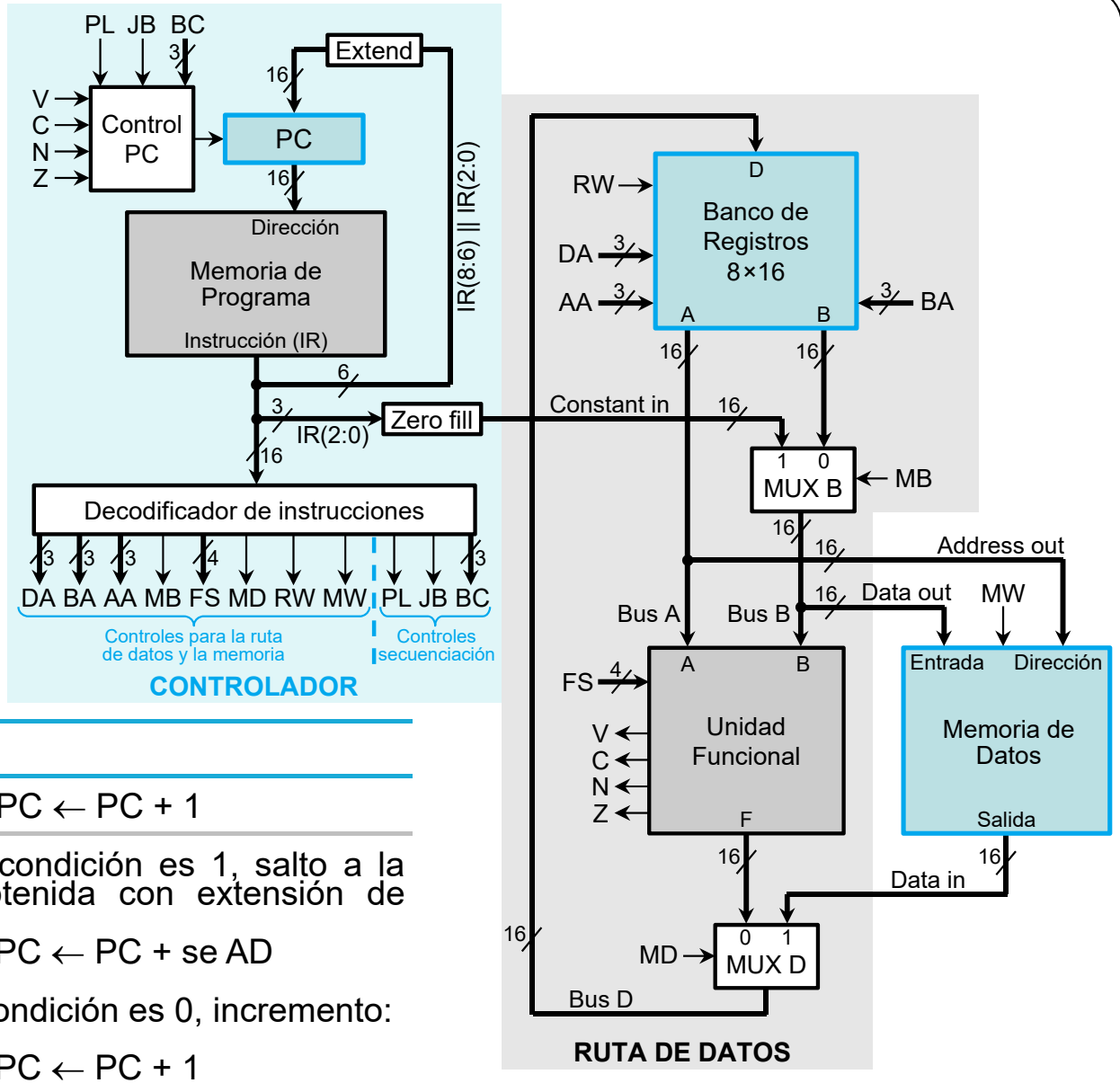


En la arquitectura Harvard, se tienen dos memorias independientes, una para las instrucciones del programa y otra para los datos.

El PC direcciona la memoria de programa, en cuyas salidas aparece la instrucción. El decodificador de instrucciones la transforma en las señales necesarias para la ruta de datos y el control de programa. Se ejecuta una instrucción en cada ciclo de reloj.

A partir de IR(2:0), "Zero fill" rellena con ceros a la izquierda para obtener "Constant in". A partir de IR(8:6)‖IR(2:0), "Extend" genera el desplazamiento para los saltos.

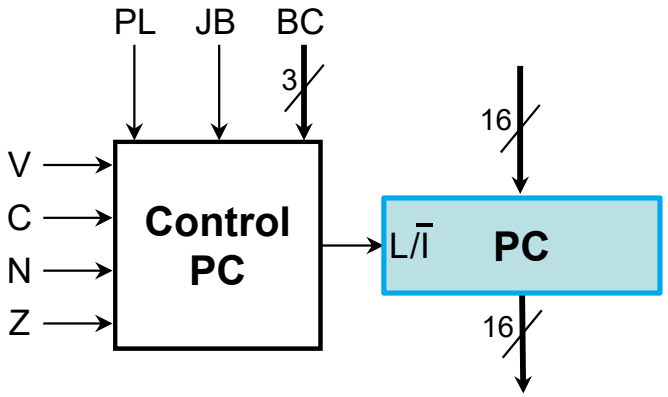
La ejecución del programa se controla mediante el combinacional "Control PC":



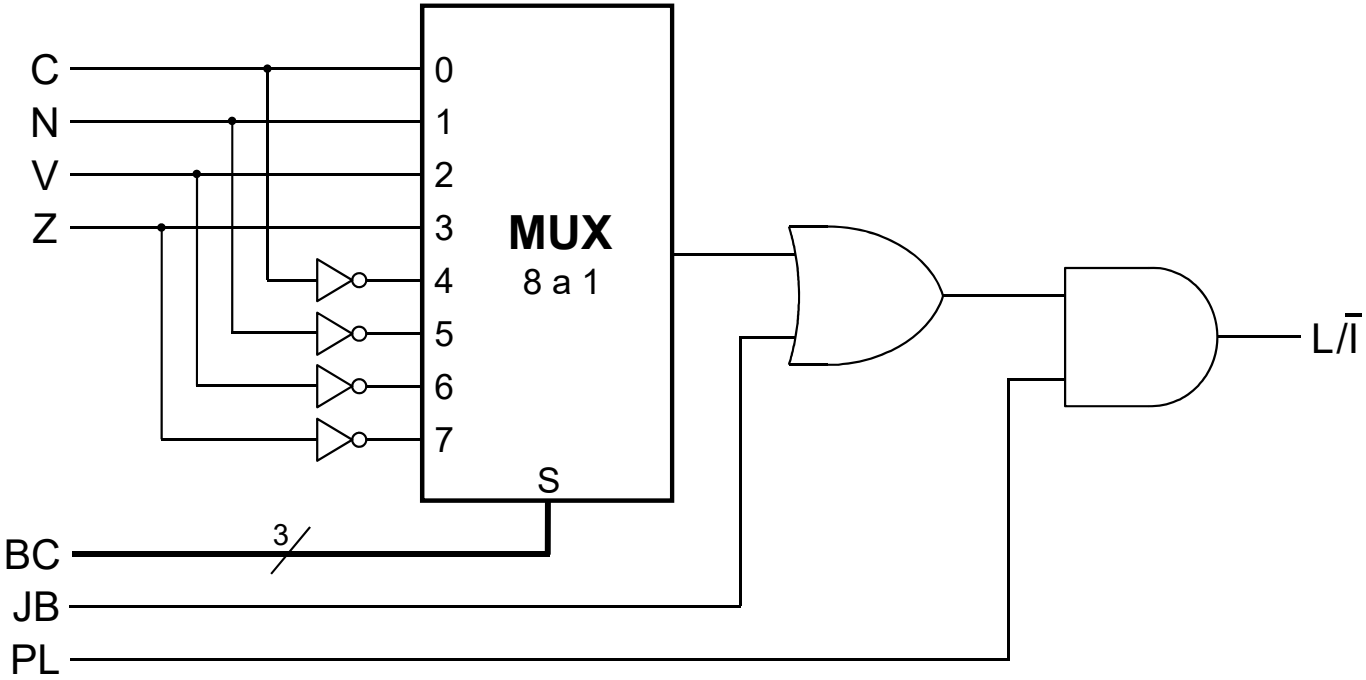
PL	JB	BC	Bit de estado	PC
0	X	XXX	X	Incremento: PC ← PC + 1
1	0	000 001 010 011 100 101 110 111	C N V Z NOT C NOT N NOT V NOT Z	Si el bit de condición es 1, salto a la dirección obtenida con extensión de signo: PC ← PC + se AD Si el bit de condición es 0, incremento: PC ← PC + 1
1	1	XXX	X	Salto incondicional: PC ← PC + se AD

CONTROL PC

PL	JB	BC	Bit de estado	PC
0	X	XXX	X	Incremento: PC ← PC + 1
1	0	000 001 010 011 100 101 110 111	C N V Z NOT C NOT N NOT V NOT Z	Si el bit de condición es 1, salto a la dirección obtenida con extensión de signo: PC ← PC + se AD Si el bit de condición es 0, incremento: PC ← PC + 1
1	1	XXX	X	Salto incondicional: PC ← PC + se AD

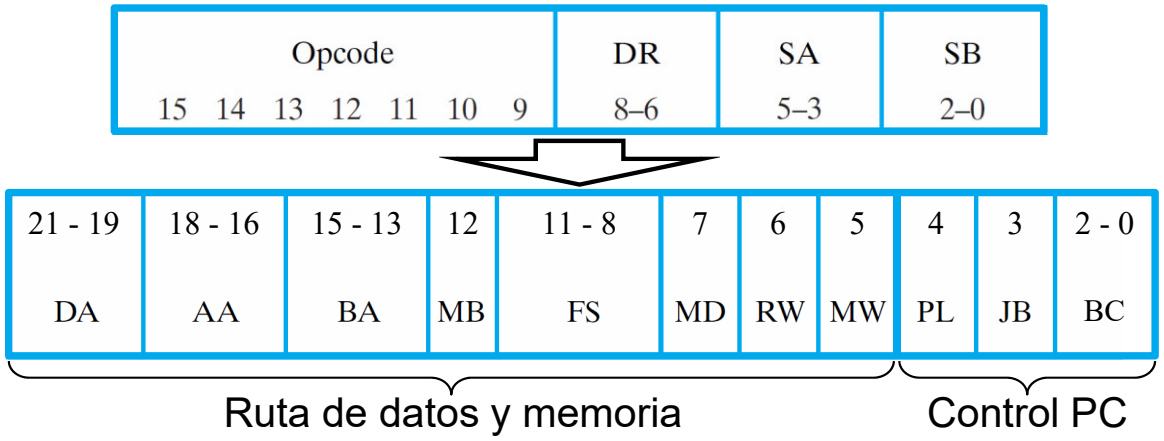


PL	JB	MUX	L/Ī
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1



DECODIFICADOR DE INSTRUCCIONES

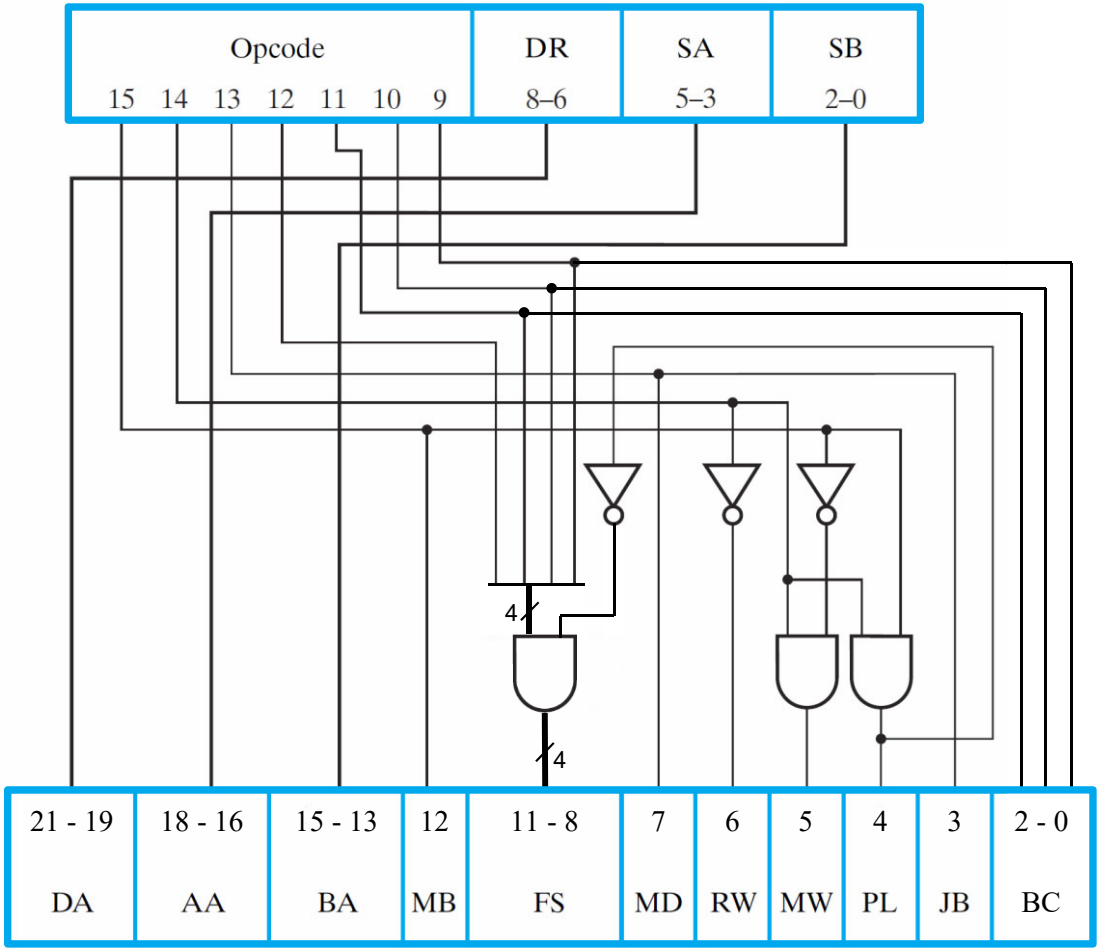
Transforma la instrucción del programa en la palabra de control:



El tipo de instrucción viene definido por los bits 13, 14 y 15 de Opcode. El resto de los bits (9 a 12) determina la operación de la unidad funcional (FS) salvo en las de salto condicional, que determinan el bit que se tiene en cuenta para las bifurcaciones (BC):

	Bits Opcode			**Bits de la palabra de control**					
Tipo de instrucción	15	14	13	MB	MD	RW	MW	PL	JB
Operaciones con la unidad funcional y registros	0	0	0	0	0	1	0	0	X
Lectura de memoria	0	0	1	X	1	1	0	0	X
Escritura en memoria	0	1	0	0	X	0	1	0	X
Operaciones con la unidad funcional y “constant in”	1	0	0	1	0	1	0	0	X
Bifurcación condicional	1	1	0	X	X	0	0	1	0
Salto	1	1	1	X	X	0	0	1	1

DECODIFICADOR DE INSTRUCCIONES



Se hace el producto lógico de FS con $\overline{PL}$ para que en las instrucciones de salto, el código de operación sea FS=0000, y así la unidad funcional simplemente transfiera el dato A y los bits de estado puedan ser evaluados.

EJEMPLOS DE INSTRUCCIONES

Instrucción	Opcode	Nemónico	Descripción	MB	FS	MD	RW	MW	PL	JB
Mueve A	000 0000	MOVA R[DR],R[SA]	R[DR] ← R[SA] *	X	0000	0	1	0	0	X
Incremento	000 0001	INC R[DR],R[SA]	R[DR] ← R[SA] + 1 *	X	0001	0	1	0	0	X
Suma	000 0010	ADD R[DR],R[SA],R[SB]	R[DR] ← R[SA] + R[SB] *	0	0010	0	1	0	0	X
Resta	000 0101	SUB R[DR],R[SA],R[SB]	R[DR] ← R[SA] – R[SB] *	0	0101	0	1	0	0	X
Decremento	000 0110	DEC R[DR],R[SA]	R[DR] ← R[SA] – 1 *	X	0110	0	1	0	0	X
AND	000 1000	AND R[DR],R[SA],R[SB]	R[DR] ← R[SA] ∧ R[SB] *	0	1000	0	1	0	0	X
OR	000 1001	OR R[DR],R[SA],R[SB]	R[DR] ← R[SA] ∨ R[SB] *	0	1001	0	1	0	0	X
XOR	000 1010	XOR R[DR],R[SA],R[SB]	R[DR] ← R[SA] ⊕ R[SB] *	0	1010	0	1	0	0	X
NOT	000 1011	NOT R[DR],R[SA]	R[DR] ← $\overline{R[SA]}$ *	X	1011	0	1	0	0	X
Mueve B	000 1100	MOVB R[DR],R[SB]	R[DR] ← R[SB] *	0	1100	0	1	0	0	X
Desp. dcha	000 1101	SHR R[DR],R[SB]	R[DR] ← sr R[SB] *	0	1101	0	1	0	0	X
Desp. izda	000 1110	SHL R[DR],R[SB]	R[DR] ← sl R[SB] *	0	1110	0	1	0	0	X
Carga inm.	100 1100	LDI R[DR],OP	R[DR] ← zf OP *	1	1100	0	1	0	0	X
Suma inm.	100 0010	ADI R[DR],R[SA],OP	R[DR] ← R[SA] + zf OP *	1	0010	0	1	0	0	X
AND inm.	100 1000	ANDI R[DR],R[SA],OP	R[DR] ← R[SA] ∧ zf OP *	1	1000	0	1	0	0	X
Carga	001 XXXX	LD R[DR],R[SA]	R[DR] ← M[R[SA]] *	X	XXXX	1	1	0	0	X
Almacena	010 XXXX	ST R[SA],R[SB]	M[R[SA]] ← R[SB] *	0	XXXX	X	0	1	0	X
Salto si cero	110 X011	BRZ R[SA],AD	Si R[SA]=0: PC←PC+se AD Si R[SA]≠0: PC←PC+1	X	0000	X	0	0	1	0
Salto si neg.	110 X001	BRN R[SA],AD	Si R[SA]<0: PC←PC+se AD Si R[SA]≥0: PC←PC+1	X	0000	X	0	0	1	0
Salto	111 XXXX	JMP AD	PC ← PC + se AD	X	XXXX	X	0	0	1	1

* Para todas estas instrucciones, PC←PC+1 también se ejecuta para apuntar a la siguiente instrucción en memoria.

EJEMPLO DE PROGRAMA

Implementaremos el algoritmo del producto visto en el Tema 1 en la máquina de ciclo sencillo. Suponemos que el multiplicando está en la posición 0 de la memoria de datos, el multiplicador en la posición 1, y que se quiere guardar el resultado en la posición 2. Multiplicando y multiplicador son números de 8 bits.

10111	Multiplicando
× 10011	Multiplicador
10111	Productos parciales
101110	
101110000	
110110101	Producto final

Direcc.	Programa		Comentarios
	LDI	R0,0	Pone a 0 el registro R0, que hará la función de acumulador.
	LDI	R1,7	Pone un 7 en R1, que hará la función de índice del bucle para la multiplicación.
	LDI	R7,0	R7 servirá de puntero para acceder a la memoria de datos. Se inicializa en 0 para que apunte a la posición donde se encuentra el multiplicando.
	LD	R2,R7	Carga el multiplicando en R2.
	INC	R7,R7	Incrementa R7 para que apunte a la posición donde se encuentra el multiplicador.
	LD	R3,R7	Carga el multiplicador en R3.
BUCLE:	ANDI	R4,R3,1	AND de R3 con la máscara 1. R4=0 sólo si el bit menos significativo de R3 es 0.
	BRZ	R4,BIT0	Si R4 es 0 salta la siguiente instrucción.
	ADD	R0,R2,R0	Acumula el multiplicando (producto parcial).
BIT0:	DEC	R1,R1	Decrementa el índice del bucle.
	BRN	R1,FIN	R1<0 sólo si hemos recorrido el bucle 8 veces, en cuyo caso salta a la posición FIN.
	SHR	R3,R3	Desplaza R3 a la derecha para acceder al siguiente bit del multiplicador.
	SHL	R2,R2	Desplaza R2 (multiplicando) a la izquierda.
	JMP	BUCLE	Vuelta al comienzo del bucle.
FIN:	INC	R7,R7	Incrementa R7 para que apunte a la posición donde se quiere almacenar el resultado.
	ST	R7,R0	Guarda el resultado final.
	END		Fin de programa (estado de espera).

ARQUITECTURA DE CICLO SENCILLO

LIMITACIONES

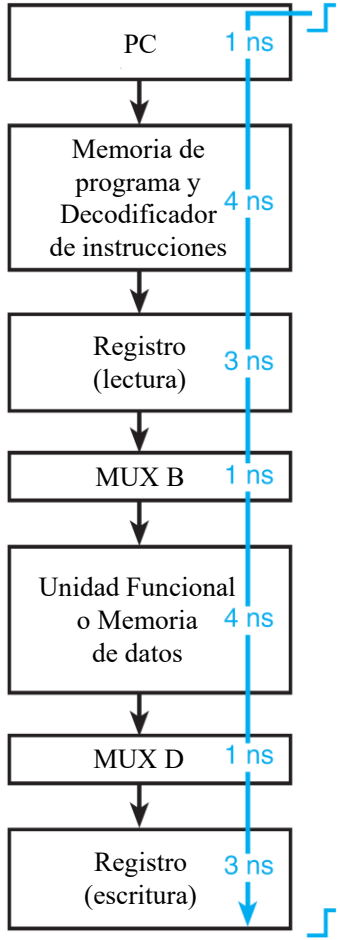
Aunque en algunos casos la estrategia de temporización y control del procesador de ciclo sencillo estudiado puede resultar útil, posee también una serie de inconvenientes:

- No se pueden llevar a cabo instrucciones complejas. Por ejemplo, una instrucción para multiplicar usando el algoritmo de suma y desplazamiento requiere más de un ciclo de reloj, dada la ruta de datos vista.

- No se puede utilizar la misma memoria para almacenar datos e instrucciones. En ese caso se necesitarían al menos dos accesos a la memoria en diferentes ciclos de reloj, uno para leer la instrucción, y otro para acceder al dato requerido.

- El período de reloj está limitado al máximo retardo de propagación en el sistema. Ejemplo:

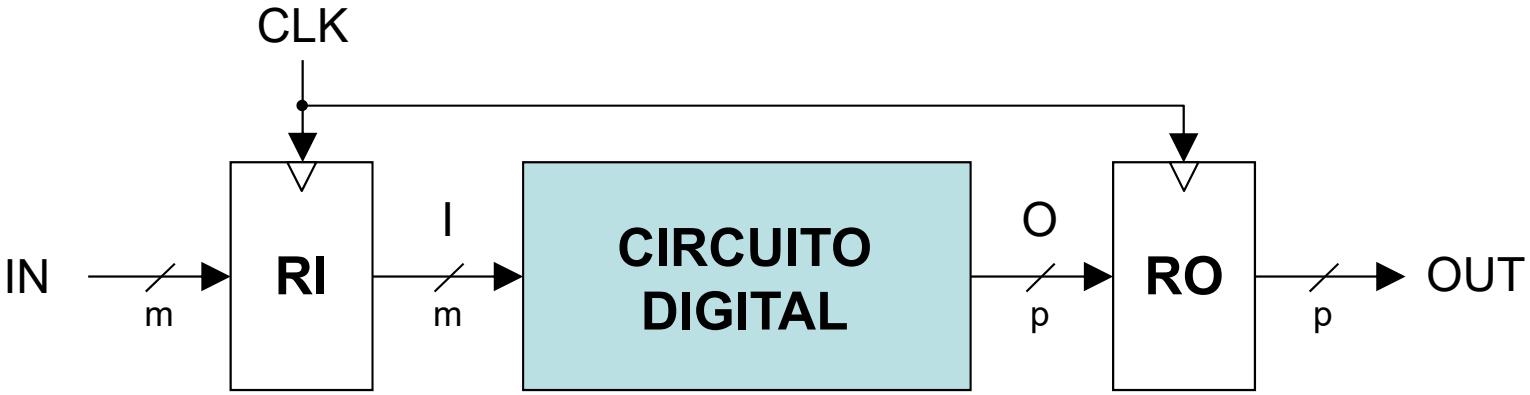
$$T_{min} = 17 \text{ ns} \Rightarrow F_{max} = 59 \text{ MHz}$$



Solución: utilizar técnicas de segmentación o recurrir a un procesador de ciclos múltiples.

PLANTEAMIENTO DEL PROBLEMA

Supongamos un circuito digital complejo insertado en un sistema de procesamiento de datos, caracterizado por un tiempo de cálculo T_{CALC}:



Despreciando los tiempos de preparación y de retardo de los registros frente a T_{CALC}, el período mínimo T_{MIN} de la señal de reloj CLK tiene que verificar:

$$T_{MIN} = T_{CALC}$$

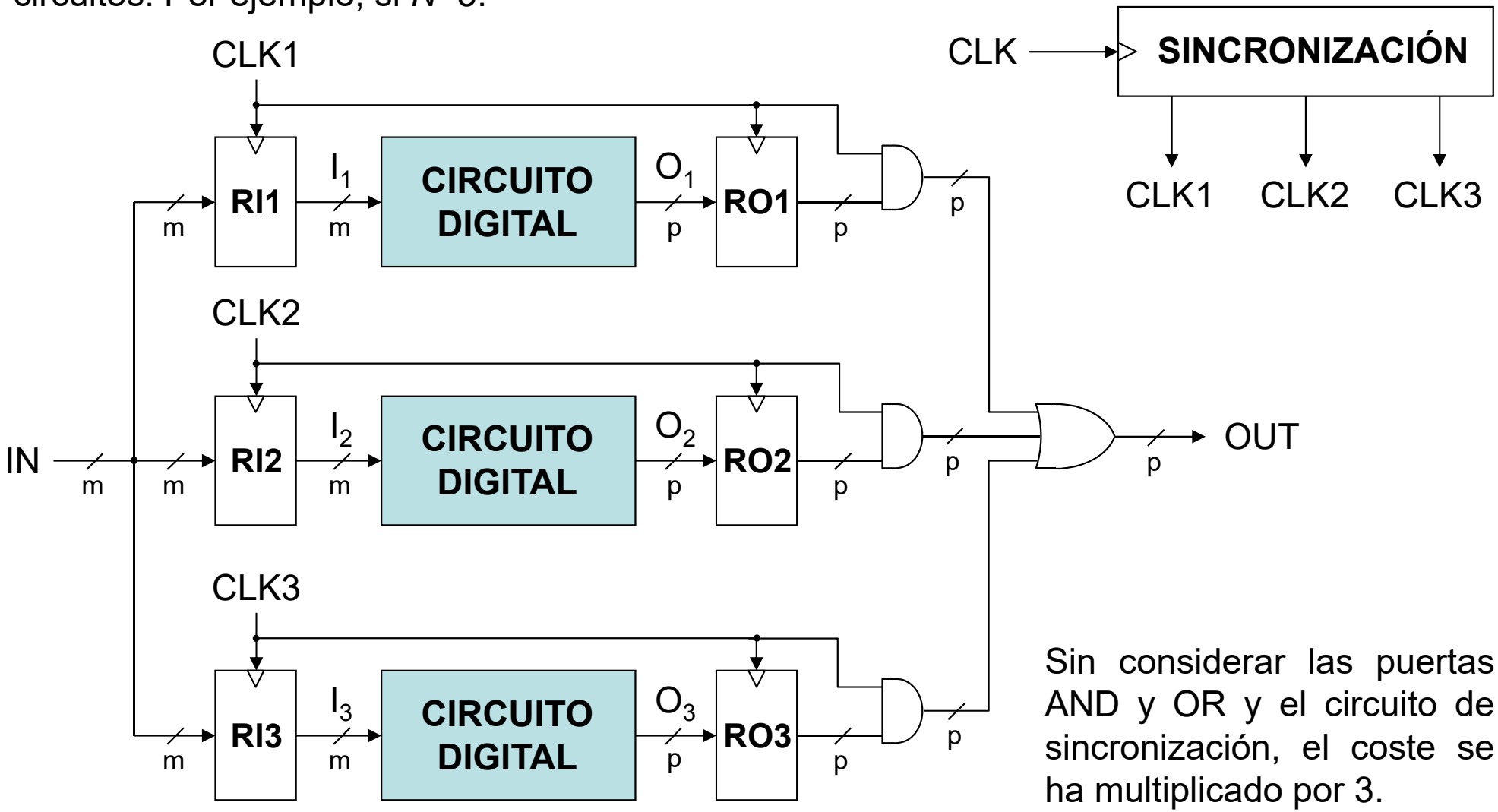
Por tanto, la frecuencia máxima de funcionamiento del sistema será:

$$F_{MAX} = 1/T_{MIN} = 1/T_{CALC}$$

¿Qué hacer si como especificación de diseño se nos pide que la frecuencia de funcionamiento del sistema sea mayor que F_{MAX}?

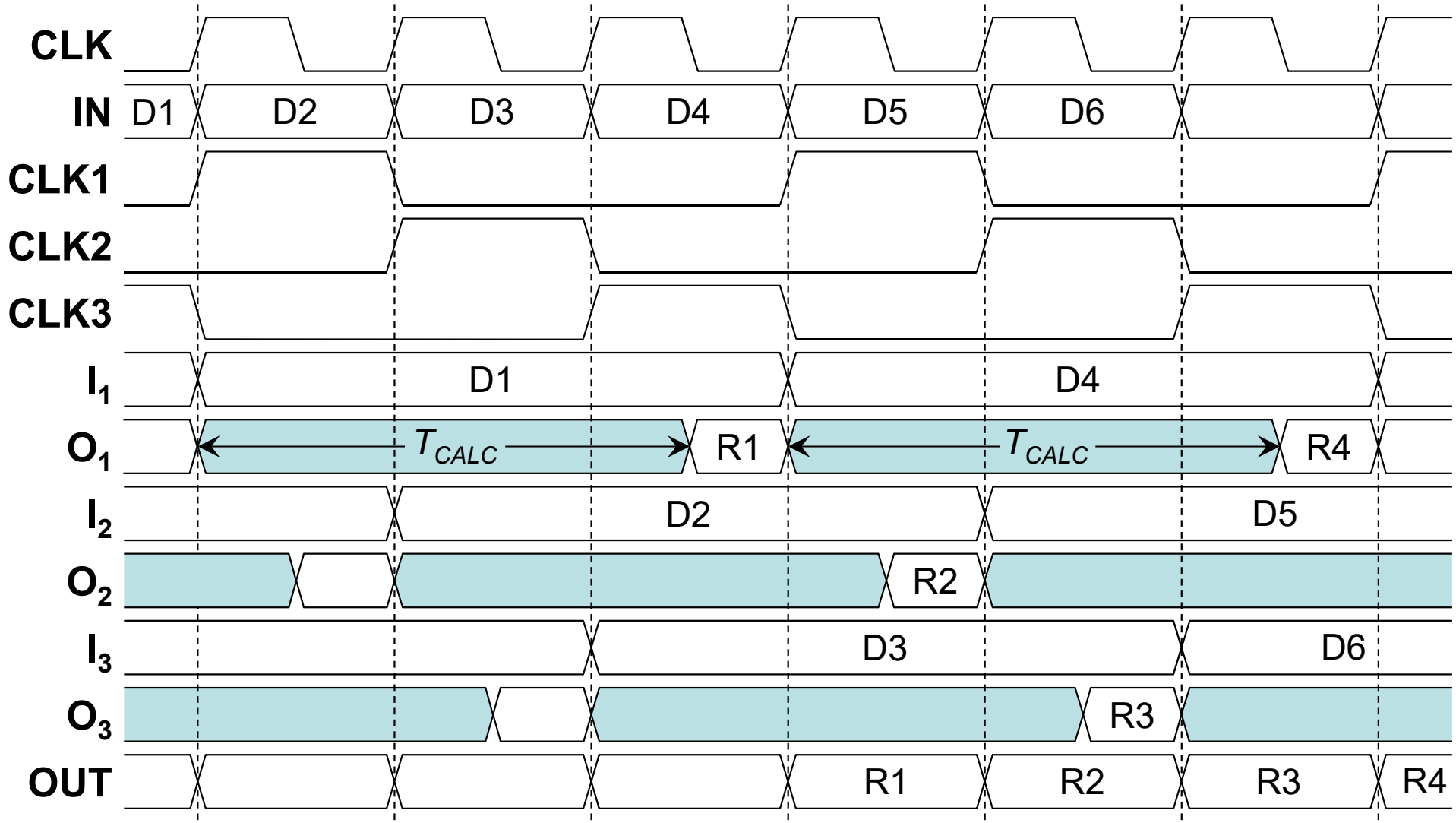
PARALELISMO

Si se desea multiplicar por *N* la frecuencia de funcionamiento, se conectarán en paralelo *N* circuitos. Por ejemplo, si *N*=3:



Sin considerar las puertas AND y OR y el circuito de sincronización, el coste se ha multiplicado por 3.

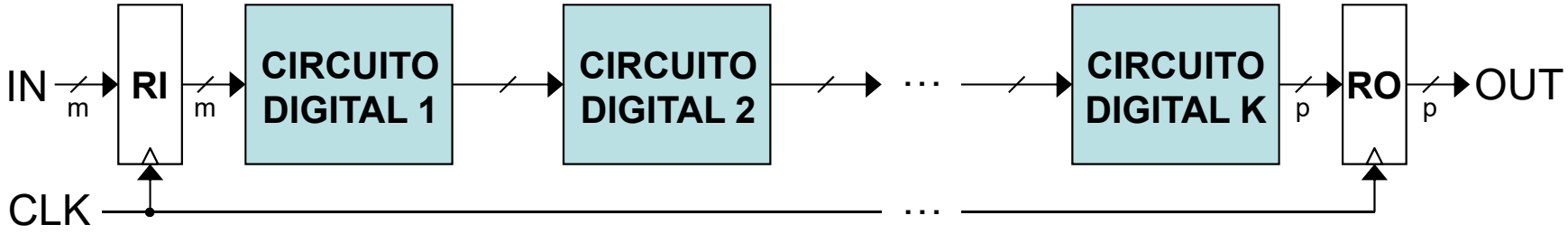
TEMPORIZACIÓN DE LAS SEÑALES



$T_{MIN} = T_{CALC}/3 \Longrightarrow$ La frecuencia máxima de funcionamiento se multiplica por 3

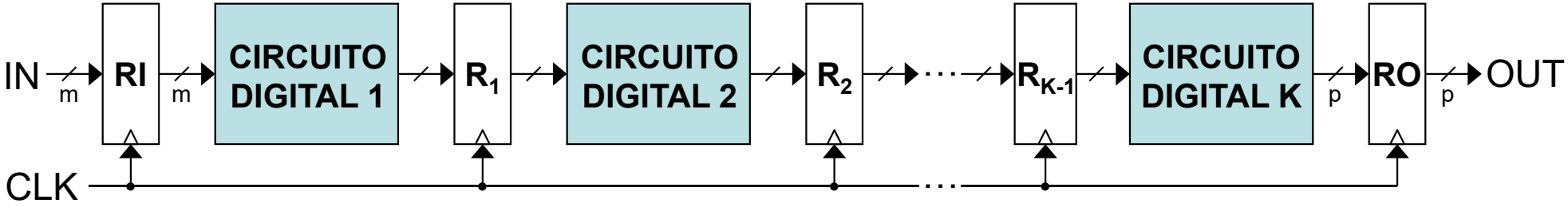
CANALIZACIÓN, SEGMENTACIÓN O "PIPELINE"

Esta técnica se puede utilizar cuando el sistema digital admite una subdivisión en circuitos más sencillos conectados en cascada:



Si el tiempo de cálculo total es T_{CALC} $\Rightarrow$ $T_{MIN} = T_{CALC}$ $\Rightarrow$ $F_{MAX} = 1/T_{CALC}$

Insertaremos registros entre cada uno de los subcircuitos para controlar el flujo de datos:



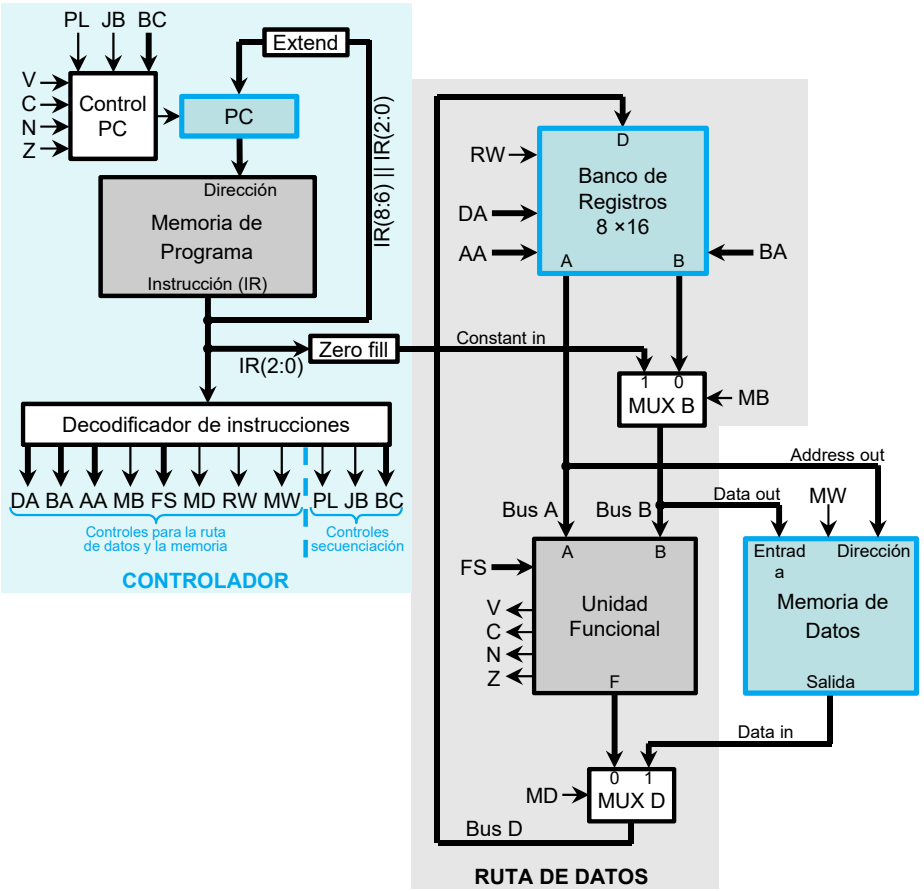
Cada subcircuito puede procesar resultados parciales que provienen de datos de origen distintos en el mismo ciclo de la señal CLK. T_{MIN} vendrá dado por el subcircuito más lento.

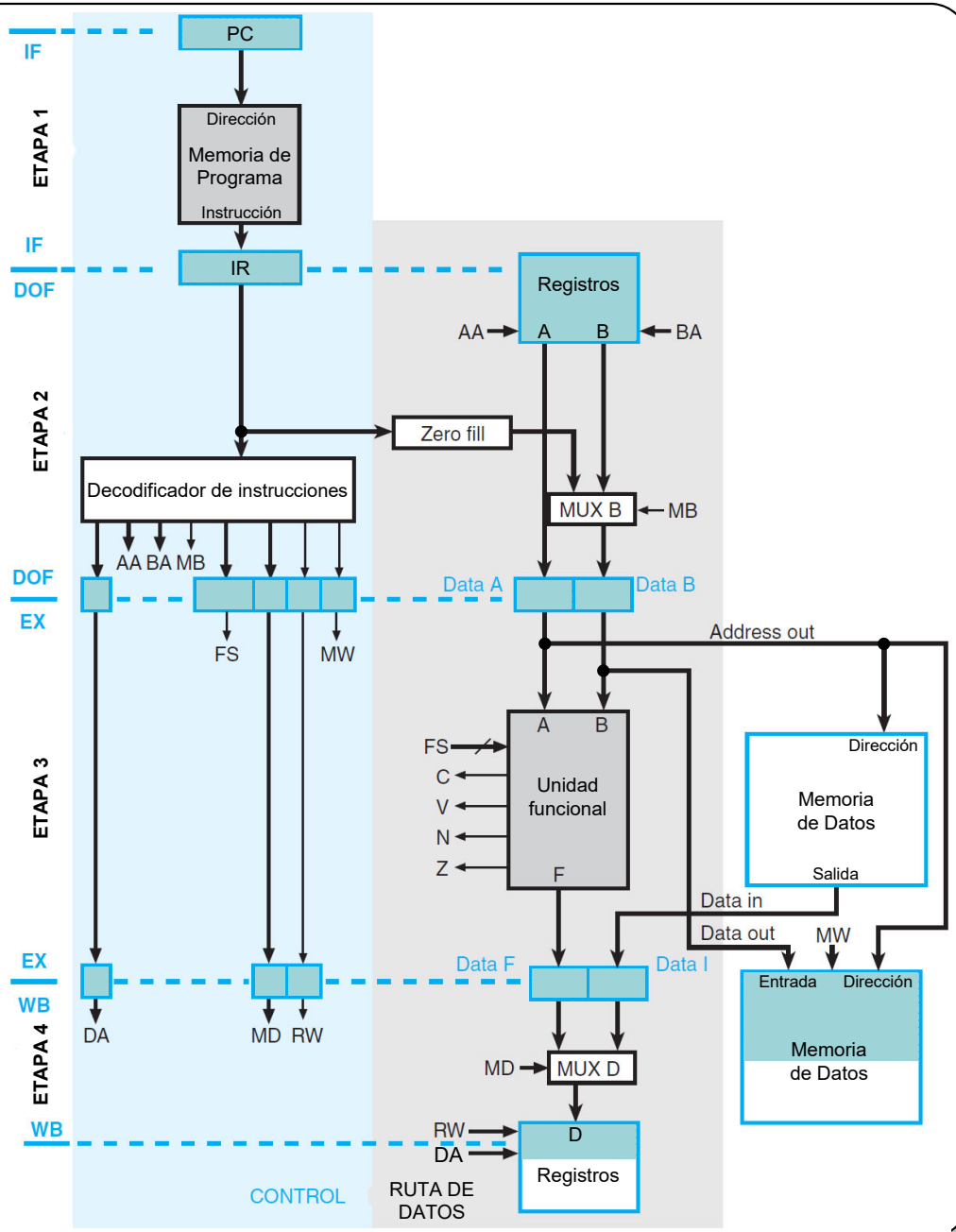
En el caso de que todos tengan el mismo tiempo de cálculo, igual a T_{CALC}/K, entonces:

$$T_{MIN} = T_{CALC}/K \quad \Rightarrow \quad F_{MAX} = K/T_{CALC}$$

La frecuencia de funcionamiento se ha multiplicado por K, mientras que el coste sólo ha aumentado ligeramente por la inclusión de los registros R_1 a R_{K-1}.

Como ejemplo de procesador segmentado, volvemos a la arquitectura de ciclo sencillo:





Etapas:

1.- Búsqueda de la instrucción (IF).

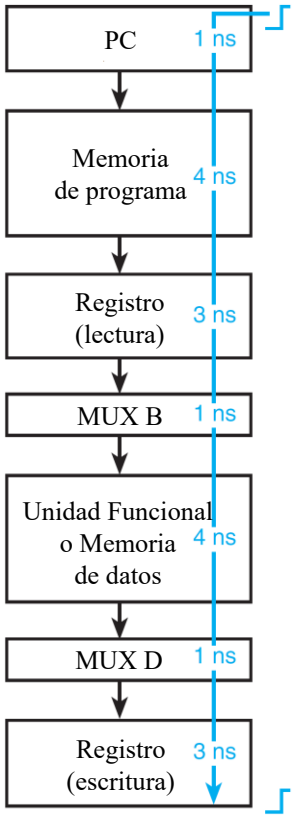
2.- Decodificación y búsqueda del dato (DOF).

3.- Ejecución (EX).

4.- Escritura del resultado (WB).

TEMPORIZACIÓN

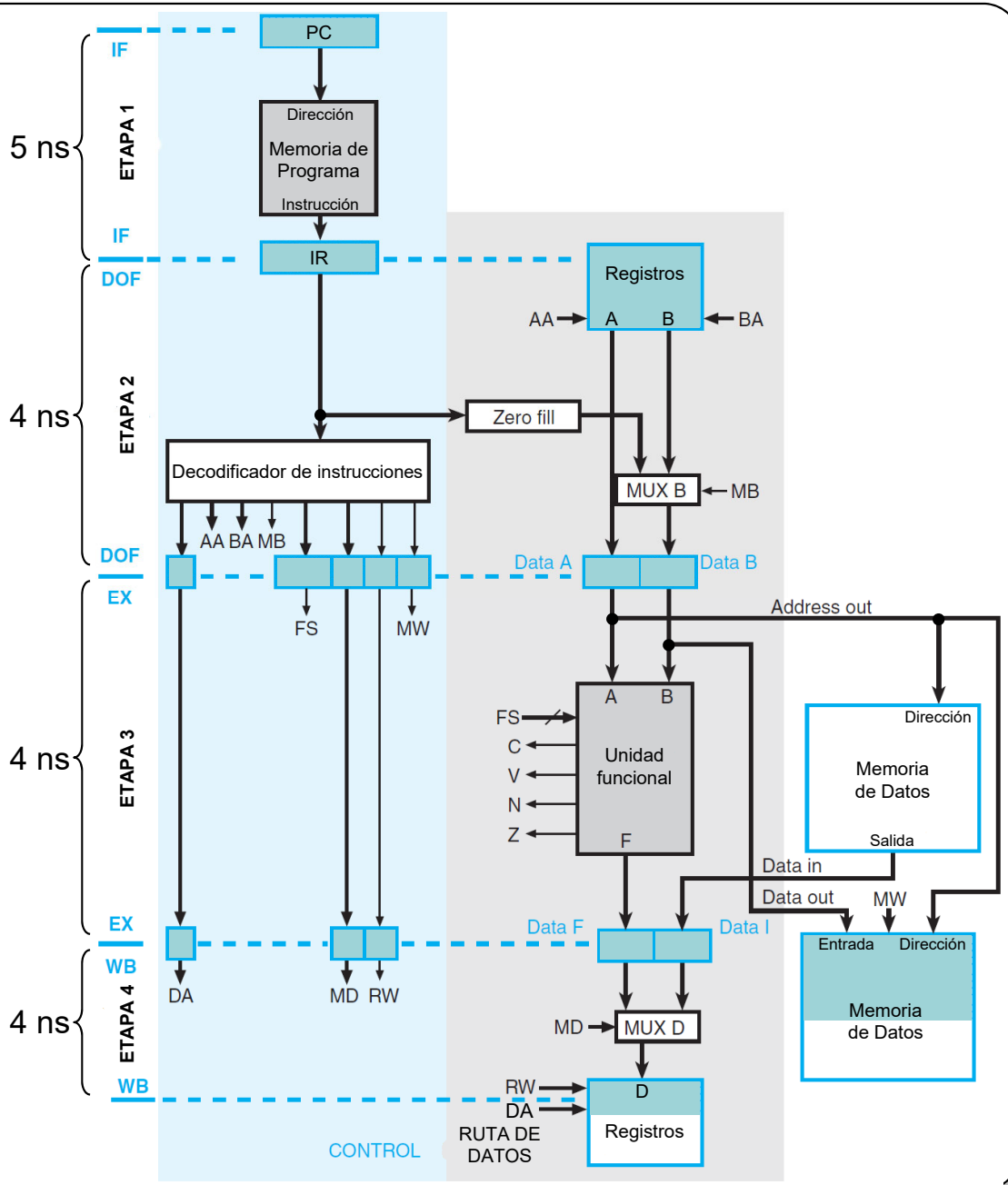
Volviendo a nuestro ejemplo:



T_{min} = 17 ns

F_{max} = 59 MHz

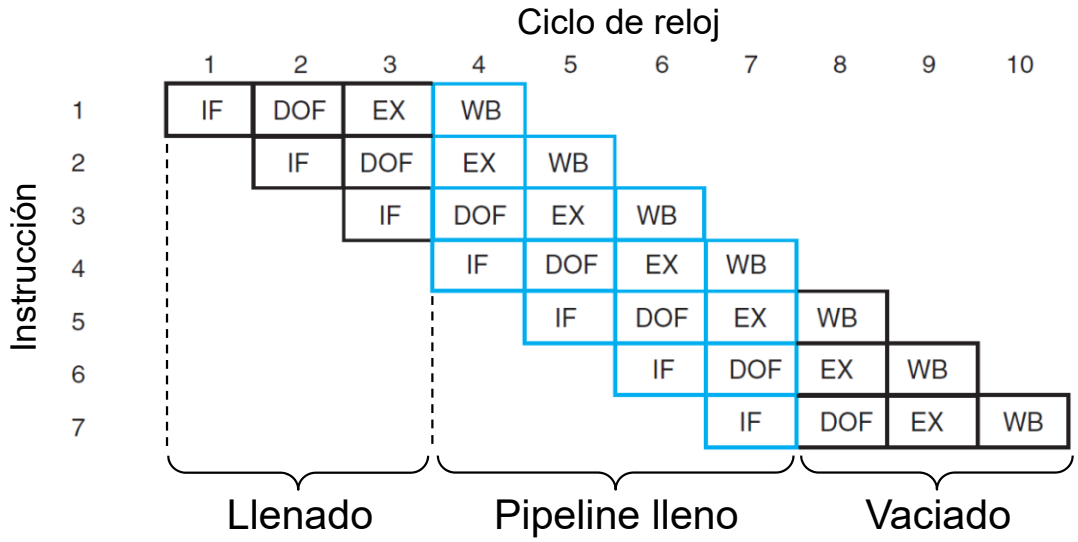
En la arquitectura segmentada, una instrucción necesita 4 ciclos de reloj, siendo 5 ns el período mínimo. Por tanto, el procesador segmentado necesita 20 ns para ejecutar una instrucción.



TEMPORIZACIÓN

La ventaja del uso de la arquitectura segmentada se obtiene cuando se ejecuta más de una instrucción. Ejemplo:

1 LDI R1, 1
2 LDI R2, 2
3 LDI R3, 3
4 LDI R4, 4
5 LDI R5, 5
6 LDI R6, 6
7 LDI R7, 7



Se necesitan 10 ciclos de reloj para ejecutar las 7 instrucciones. Cuando el pipeline está totalmente lleno, se ejecuta una instrucción en cada ciclo de reloj:

$$\text{Rendimiento} = \frac{\text{Tiempo de ejecución en la arquitectura sin segmentar}}{\text{Tiempo de ejecución en la arquitectura segmentada}} = \frac{17\ \text{ns}}{5\ \text{ns}} = 3.4$$

Aunque el pipeline tiene 4 etapas, el tiempo de cálculo no se divide exactamente por 4 debido a que los retardos no son exactamente iguales en cada etapa.

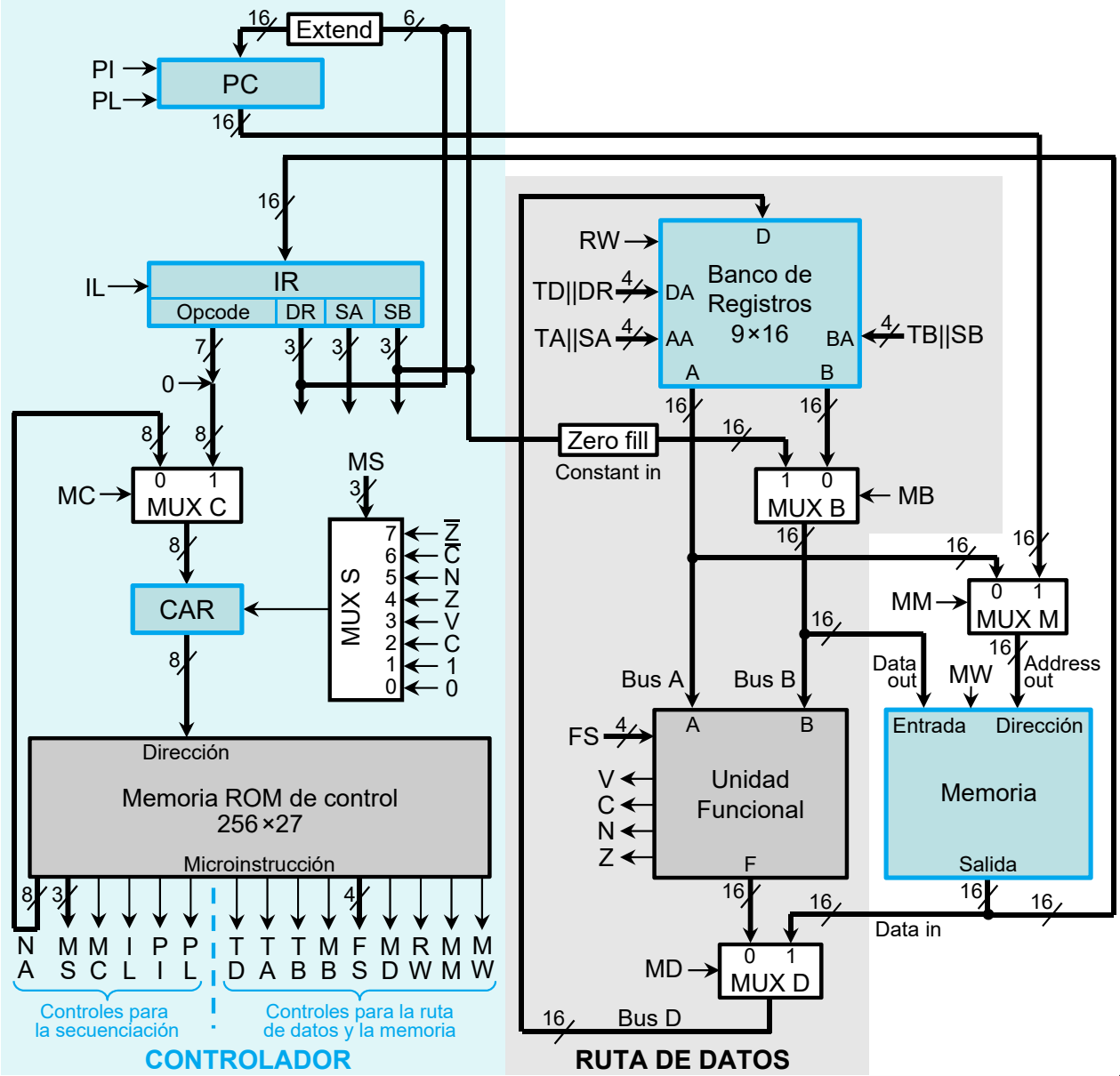
El rendimiento del pipeline baja cuando se producen saltos y bifurcaciones en el programa, debido a que en esos casos hay que vaciar el pipeline y volverlo a llenar.

TEMA 2: SISTEMAS ALGORÍTMICOS PROGRAMABLES

2.6.- Control microprogramado de ciclos múltiples – Arquitectura Von-Neumann

En la arquitectura Von-Neumann se usa una sola memoria para programa y datos. Implementaremos instrucciones de más de un ciclo de reloj:

- A la memoria se accede desde el PC para buscar las instrucciones, y a través del bus A para buscar los datos. Se necesita el MUX M.
- Para mantener la instrucción durante varios ciclos se almacena en el *registro de instrucción* IR.
- PC puede ser incrementado, cargado con PC + se AD para los saltos, o mantener su contenido.
- DR, SA y SB van directamente al banco de registros.
- Para cálculos internos del procesador, se introduce en el banco un registro R8 accesible con TA, TB y TD en 1.
- Las microinstrucciones se programan en una memoria ROM direccionada por el registro CAR. CAR se incrementa o se carga en función del MUX S. Con MUX C se selecciona si la carga se hace con el Opcode más un bit 0 a la izquierda, o con la palabra NA de 8 bits.



MICROINSTRUCCIONES

Formato de la microinstrucción:

26 25 24 23 22 21 20 19	18 17 16	15	14	13	12	11	10	9	8	7 6 5 4	3	2	1	0
NA	MS	MC	IL	PI	PL	TD	TA	TB	MB	FS	MD	RW	MM	MW

Terminales de control para la secuenciación de la unidad de control:

MS			**MC**	**IL**	**PI**	**PL**	
Action	**Select**	**Code**	**Select**	**Action**	**Action**	**Action**	**Code**
Increment CAR	CNT	000	NA (NXA)	No load (NLI)	No load (NLP)	No load (NLP)	0
Load CAR	NXT	001	Opcode (OPC)	Load instr. (LDI)	PC ← PC+1 (INP)	Load PC (LDP)	1
If $C = 1$ load CAR else CAR←CAR+1	BC	010					
If $V = 1$ load CAR else CAR←CAR+1	BV	011					
If $Z = 1$ load CAR else CAR←CAR+1	BZ	100					
If $N = 1$ load CAR else CAR←CAR+1	BN	101					
If $C = 0$ load CAR else CAR←CAR+1	BNC	110					
If $Z = 0$ load CAR else CAR←CAR+1	BNZ	111					

MICROINSTRUCCIONES

Formato de la microinstrucción:

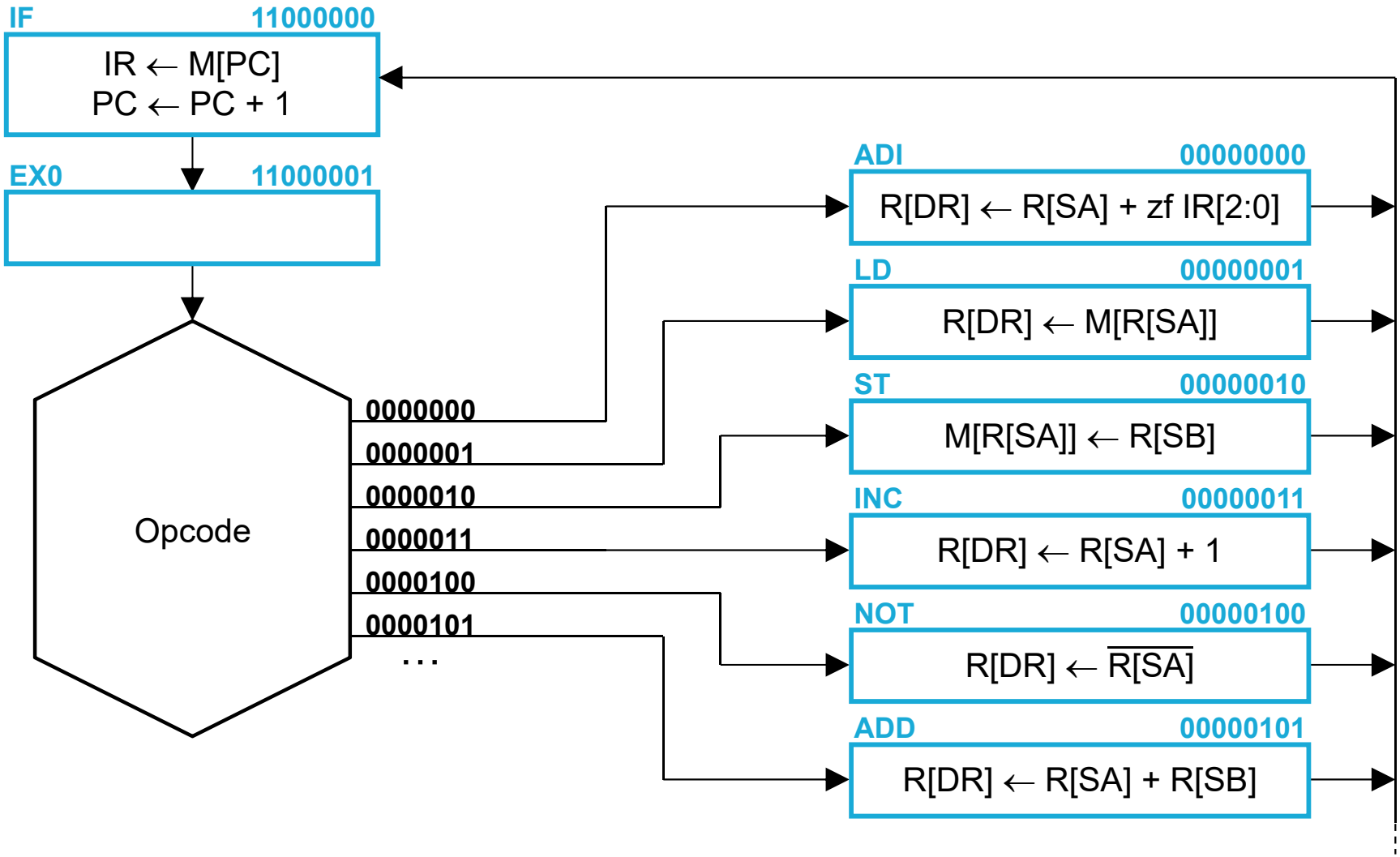
26 25 24 23 22 21 20 19	18 17 16	15	14	13	12	11	10	9	8	7 6 5 4	3	2	1	0
NA	MS	MC	IL	PI	PL	TD	TA	TB	MB	FS	MD	RW	MM	MW

Terminales de control para la ruta de datos:

TD Select	TA Select	TB Select	MB Select	Code	FS Function	Code	MD Select	RW Action	MM Select	MW Action	Code
DR	SA	SB	Register	0	$F = A$	0000	FnUt	No write (NW)	Address (MA)	No Write (NW)	0
R8	R8	R8	Constant	1	$F = A+1$	0001	Data	Write (WR)	PC	Write (WR)	1
					$F = A+B$	0010					
					$F = A+B+1$	0011					
					$F = A+\overline{B}$	0100					
					$F = A+\overline{B}+1$	0101					
					$F = A-1$	0110					
					$F = A$	0111					
					$F = A \wedge B$	1000					
					$F = A \vee B$	1001					
					$F = A \oplus B$	1010					
					$F = \overline{A}$	1011					
					$F = B$	1100					
					$F = \mathrm{sr}\,A$	1101					
					$F = \mathrm{sl}\,A$	1110					

MICROPROGRAMACIÓN

Los *microprogramas* suministran la información necesaria en la memoria de control para buscar (*fetch*) y ejecutar (*execute*) las instrucciones:



MICROPROGRAMACIÓN

Microprograma simbólico para la búsqueda y la ejecución de las seis instrucciones:

Address	NA	MS	MC	IL	PI	PL	TD	TA	TB	MB	FS	MD	RW	MM	MW
IF	—	CNT	—	LDI	INP	NLP	—	—	—	—	—	—	NW	PC	NW
EX0	—	NXT	OPC	NLI	NLP	NLP	—	—	—	—	—	—	NW	—	NW
ADI	IF	NXT	NXA	NLI	NLP	NLP	DR	SA	—	Constant	$F = A + B$	FnUt	WR	—	NW
LD	IF	NXT	NXA	NLI	NLP	NLP	DR	SA	—	—	—	Data	WR	MA	NW
ST	IF	NXT	NXA	NLI	NLP	NLP	—	SA	SB	Register	—	—	NW	MA	WR
INC	IF	NXT	NXA	NLI	NLP	NLP	DR	SA	—	—	$F = A + 1$	FnUt	WR	—	NW
NOT	IF	NXT	NXA	NLI	NLP	NLP	DR	SA	—	—	$F = \overline{A}$	FnUt	WR	—	NW
ADD	IF	NXT	NXA	NLI	NLP	NLP	DR	SA	SB	Register	$F = A + B$	FnUt	WR	—	NW

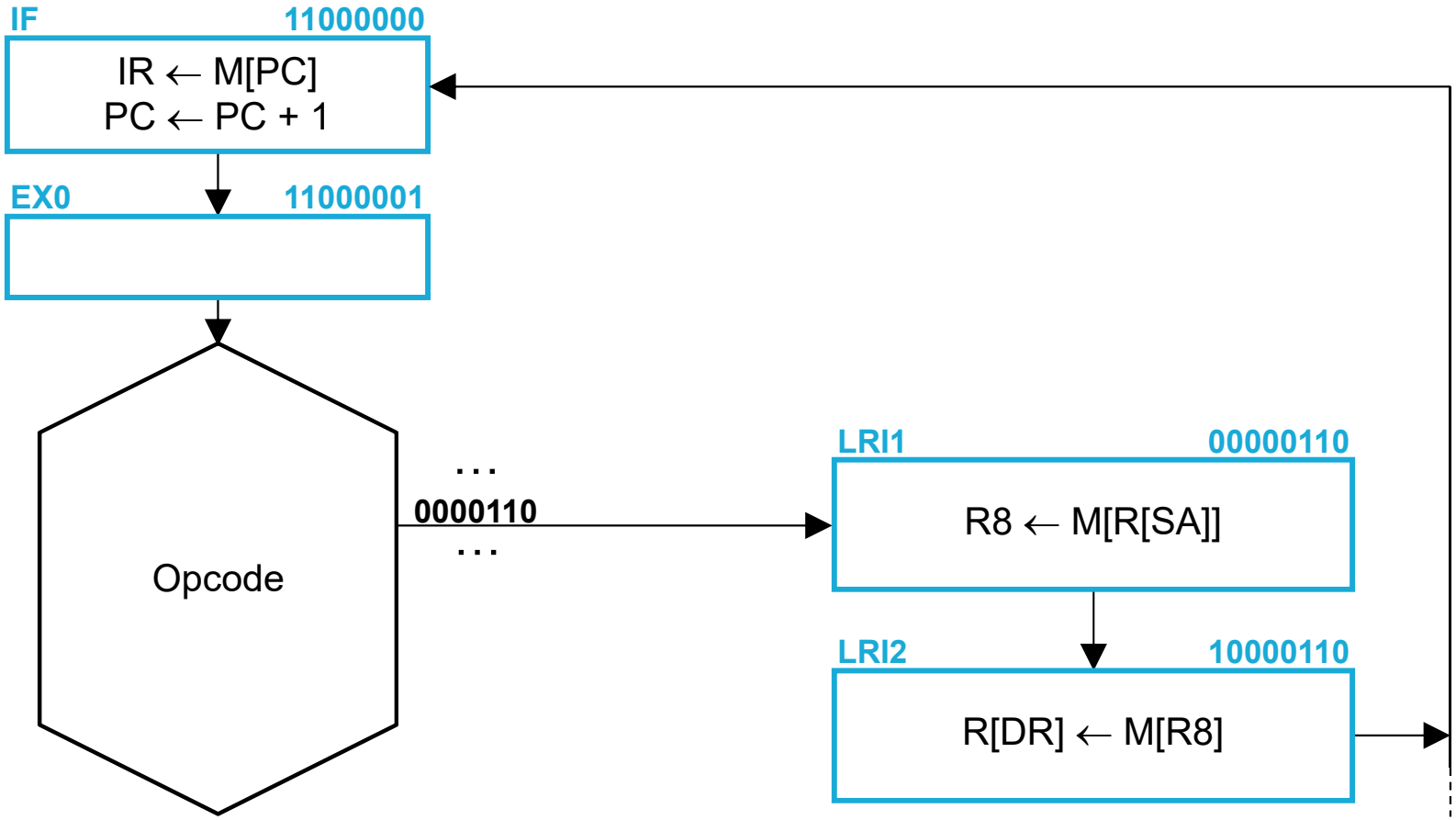
Microprograma en binario para la búsqueda y la ejecución de las seis instrucciones:

Address	NA	MS	MC	IL	PI	PL	TD	TA	TB	MB	FS	MD	RW	MM	MW
C0	XX	000	X	1	1	0	X	X	X	X	XXXX	X	0	1	0
C1	XX	001	1	0	0	0	X	X	X	X	XXXX	X	0	X	0
00	C0	001	0	0	0	0	0	0	X	1	0010	0	1	X	0
01	C0	001	0	0	0	0	0	0	X	X	XXXX	1	1	0	0
02	C0	001	0	0	0	0	X	0	0	0	XXXX	X	0	0	1
03	C0	001	0	0	0	0	0	0	X	X	0001	0	1	X	0
04	C0	001	0	0	0	0	0	0	X	X	1011	0	1	X	0
05	C0	001	0	0	0	0	0	0	0	0	0010	0	1	X	0

Los campos *Address* y *NA* están expresados en hexadecimal.

MICROPROGRAMACIÓN

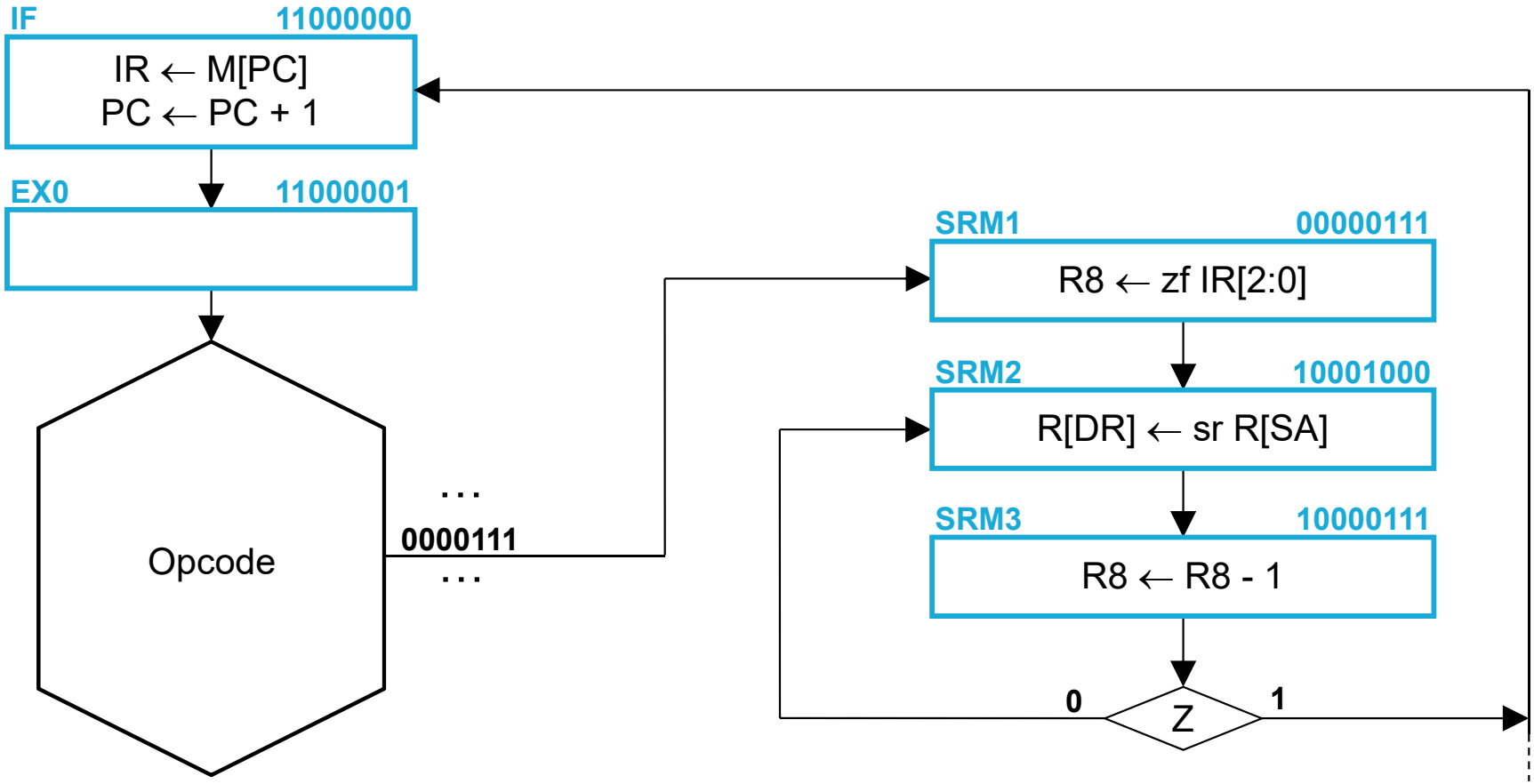
Veremos ahora dos ejemplos de instrucciones que no son posibles en el procesador de ciclo sencillo. La primera es la de *carga indirecta de registro* (LRI):



LRI requiere 4 ciclos de reloj. Hacer lo mismo sin ella requeriría 2 instrucciones LD, es decir, 6 ciclos de reloj. Por tanto, LRI implica una mejora en cuanto al tiempo de ejecución y espacio en memoria.

MICROPROGRAMACIÓN

El segundo ejemplo es un *desplazamiento múltiple a la derecha* (SRM), en el que se toma la palabra contenida en R[SA], se desplaza a la derecha un número *s* de bits dado en el campo OP (IR[2:0]), y el resultado queda almacenado en R[DR]:



SRM requiere 2*s*+3 ciclos de reloj. Hacer lo mismo a partir de desplazamientos individuales requeriría 3*s* ciclos. Por tanto, SRM implica una mejora de *s*-3 ciclos en cuanto al tiempo de ejecución, aparte de un considerable ahorro de espacio en memoria.

MICROPROGRAMACIÓN

Microprogramas simbólico y binario para las operaciones LRI y SRM:

Address	NA	MS	MC	IL	PI	PL	TD	TA	TB	MB	FS	MD	RW	MM	MW
IF	—	CNT	—	LDI	INP	NLP	—	—	—	—	—	—	NW	PC	NW
EX0	—	NXT	OPC	NLI	NLP	NLP	—	—	—	—	—	—	NW	—	NW
LRI1	LRI2	NXT	NXA	NLI	NLP	NLP	R8	SA	—	—	—	Data	WR	MA	NW
LRI2	IF	NXT	NXA	NLI	NLP	NLP	DR	R8	—	—	—	Data	WR	MA	NW
SRM1	SRM2	NXT	NXA	NLI	NLP	NLP	R8	—	—	Constant	$F = B$	FnUt	WR	—	NW
SRM2	SRM3	NXT	NXA	NLI	NLP	NLP	DR	SA	—	—	$F = \text{sr}\ A$	FnUt	WR	—	NW
SRM3	IF	BZ	NXA	NLI	NLP	NLP	R8	R8	—	—	$F = A\text{-}1$	FnUt	WR	—	NW

Address	NA	MS	MC	IL	PI	PL	TD	TA	TB	MB	FS	MD	RW	MM	MW
C0	XX	000	X	1	1	0	X	X	X	X	XXXX	X	0	1	0
C1	XX	001	1	0	0	0	X	X	X	X	XXXX	X	0	X	0
06	86	001	0	0	0	0	1	0	X	X	XXXX	1	1	0	0
86	C0	001	0	0	0	0	0	1	X	X	XXXX	1	1	0	0
07	88	001	0	0	0	0	1	X	X	1	1100	0	1	X	0
88	87	001	0	0	0	0	0	0	X	X	1101	0	1	X	0
87	C0	100	0	0	0	0	1	1	X	X	0110	0	1	X	0

Los campos *Address* y *NA* están expresados en hexadecimal.